

Deep Dive: Building Your First Pipeline

Palantir Foundry 数据管线深度课程 · 按左侧导航顺序完整收录 (25 个 lesson, 含图片)

抓取日期: 2026-09-08 · 来源: learn.palantir.com

课程总览

课程名称: Deep Dive: Building Your First Pipeline

来源链接: <https://learn.palantir.com/deep-dive-building-your-first-pipeline>

目录（按左侧导航顺序）

1. Introduction
2. Permissions & Help
3. Introduction
4. Create a Foundry Learning Project
5. Create a Course-Specific Training Folder
6. Introduction
7. Download the Data Sources
8. [Optional] Deploy Datasets via Marketplace
9. Create a New Pipeline
10. Basic Transforms: Clean the Products Dataset
11. Advanced Transforms: Clean the Customers Dataset
12. Work with Different File Types: Preprocess and Clean the Transactions Dataset
13. Introduction
14. Simple Joins: Join Transactions and Products
15. Chaining Joins: Join Transactions x Products with Customers
16. Add a Dataset Output
17. Identify a Bad Join: Drill into Transactions x Products
18. Create an Operationally Valuable Dataset
19. Introduction
20. Other Pipeline Builder Capabilities
21. User-Defined Functions (UDFs)
22. Best Practices: Segmenting your Pipeline and Materializing Outputs
23. Pipeline Maintainability
24. Summary
25. [Optional] Course Feedback

Introduction

Lesson 1 / 25 · [原文链接](#)

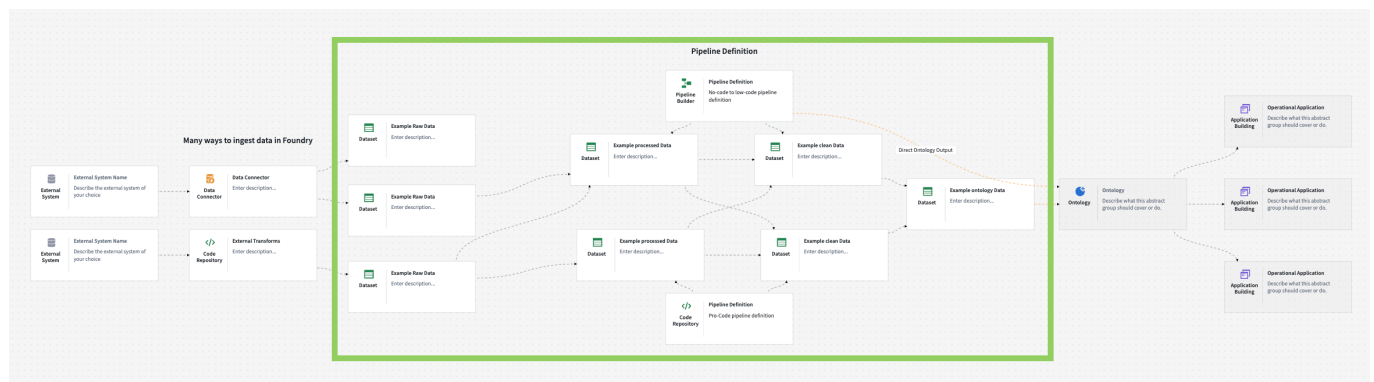
This course is continuously updated. If you have feedback, please email training-feedback@palantir.com.

In the next 60 minutes, you will learn how to build your first data pipeline in Foundry's Pipeline Builder.

What is a data pipeline?

A data pipeline is a series of data processing steps that systematically transform raw data into valuable insights. Data pipelines clean, process, and integrate data, powering data-driven analyses and decision making within an organization.

An example end-to-end data flow in Foundry could look something like the below diagram, where the outputs of your data pipeline feed directly into the Ontology to power operational applications.



Why are data pipelines important?

In Foundry, pipelines are the backbone of the flow and transformation of data from its source form (raw) into a state that is operationally valuable. Mastering pipelines is mastering the art of turning data into actionable intelligence. It's the difference between having a sea of indecipherable data points and a well-organized dataset ready for strategic use.

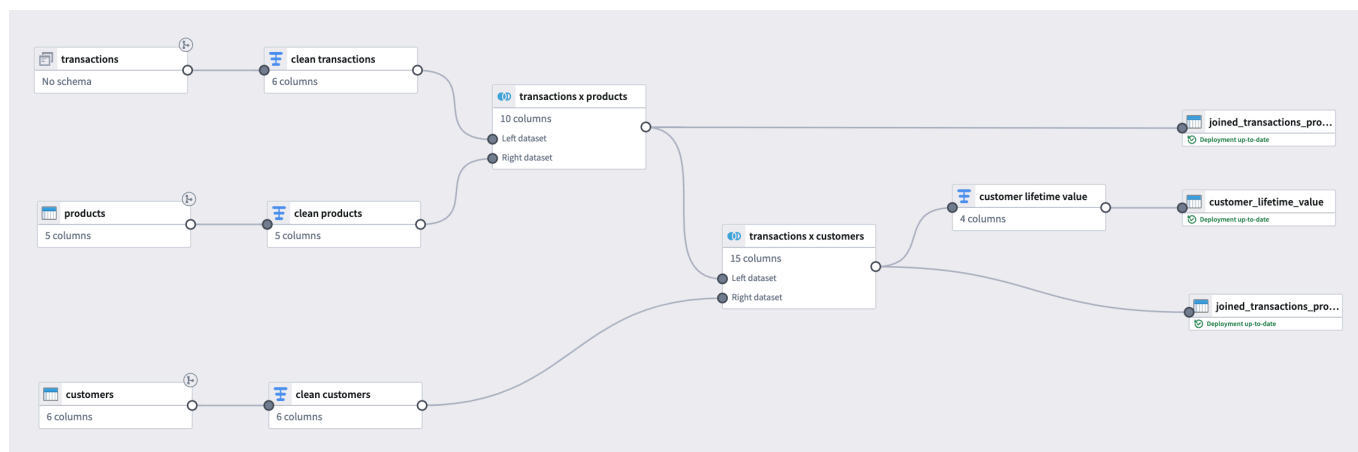
What is Pipeline Builder?

Pipeline Builder is Foundry's key low-code/no-code tool for effectively creating and managing of data pipelines. With Pipeline Builder, you can construct robust data processing workflows without the need for extensive coding. It's an intuitive interface that provides access to a suite of pre-built Spark modules, enabling you to tackle common data challenges with ease.

The Training

This training will guide you through the creation of a pipeline and the logic that powers it in Pipeline Builder. By the end of this session, you'll be equipped to take data from its raw state and transform it into something

operationally valuable. The training will also briefly touch on surrounding aspects of pipeline construction — such as branching, builds, schedules, and data validation. By the end of the training, you would have built a fully functional pipeline with minimal, if any, coding requirements, similar to the below:



Our partners at Ontologize have developed [a step-by-step YouTube video](#) of this course. Feel free to use the video to follow along as you complete the course material.

Permissions & Help

Lesson 2 / 25 · [原文链接](#)

As you work through this course, you may encounter certain issues. For example, there may be specific features, limitations, or security controls in place on your Foundry instance that prevent you from following these generic instructions.

Depending on the issue that you encounter, here is how to get help!

Training Application

Your organization's Foundry Platform Administrator may have posted local documentation within Foundry's **Training** application. This will typically cover any user restrictions that exist in your specific Foundry enrollment regarding creating projects, uploading data, creating or editing objects, or deploying Marketplace bundles. It may also include organization-specific training content.

You can open the Training app from the bottom of Foundry’s left navigation bar by selecting **Support > Training**.

Permissions

To complete this specific learning course, you will need access to the following platform components:

- Workspace Navigator (Compass)
- Pipeline Builder

Users will need the following permissions in order to run all steps of this learning course:

Permissions Required	Workaround if permissions are not available
Create Learning Project	Platform Admin team will need to create Learning Project for each user.
Edit Learning Project	Platform Admin team will need to create Learning Course Folder and give editor access.
Upload Files	Platform Admin will need to Upload Files and provide instructions to copy them to the Learning Course Folder.
Create Pipeline Builder Resources	None. Permissions are required.

Documentation

You can find out more about any of the Foundry applications or concepts presented in this course by searching in the Foundry documentation: <https://www.palantir.com/docs/foundry>

Screenshots and Instructions

Our platform is evolving quickly. The screenshots in this course may differ slightly from how Foundry appears on your screen. In most cases, these differences should be minor and cosmetic, and it should still be possible to continue with the course. If you notice such differences or are unable to continue because of them, please report them to training-feedback@palantir.com.

Feedback

This training is being actively developed and we'd love to hear your feedback! If you have comments or suggestions, or if you encounter issues you cannot resolve by any of the means listed above, please reach out to training-feedback@palantir.com.

Introduction

Lesson 3 / 25 · [原文链接](#)

All of the resources you create in Foundry need to live inside a Project. For production use cases, these resources likely need to be shared across your organization and it is recommended you create a Project for each stage of the workflow, as noted in our [online documentation](#).

During this learning course, you will be creating resources that will be specific to your learning experience. Depending on how your environment is set up, you will want to use a naming convention to easily distinguish your resources from those of other users. We will use the convention `<username>` throughout this course to represent your user login id. Wherever you see `<username>` please replace with your own user login id.

We advise using a single Foundry Project to store all your training materials for all courses on this site. You should then create a dedicated Folder within your training Project for each learning course. If your organization prefers a different Project set-up convention, you may need to adapt some instructions. Contact your platform support team for more details.

Create a Foundry Learning Project

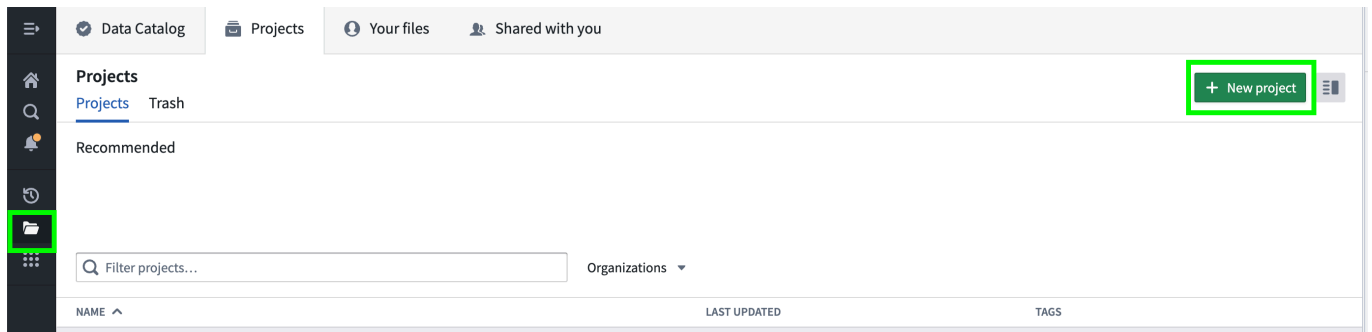
Lesson 4 / 25 · [原文链接](#)

Note if you already have your own Learning Project on your Foundry instance you can skip this step and move on to the next step to create your course-specific folder.

Step 1: Create a Project

1. Click on the Files icon on the left sidebar
2. Click on New Project in the top right

Note: If you do not have permissions to create a project, please select a project you feel comfortable using and continue with the next instructions to create a folder. Do not use "Your files" nor your home folder as certain tools cannot be accessed from your home folder.



Step 2: Configure your Project

1. Give your project a name, such as `Learning (<username>)`
2. Give your project a description that would help another user understand what the project contains and is used for
3. Select your organization's Namespace from the dropdown (*Tip: If you don't see this dropdown option then you don't need to do it*)
4. Set the Default role to **Discover**.
5. Click on **Create**.

 Create new project ✕

Name

learning (jsmith)

Description (optional)

This project contains folders for each learning course for the user jsmith.

Namespace ?

Training ▼

Organizations · Any of

 Training ▼

Default role

Discoverer ▼

Everyone from  Palantir can see the existence of this project and is granted the Discoverer role.

Cancel

Create

(Tip: Right-click on the image and click "Open Image in New Tab" to see the full-size version)

Learn more

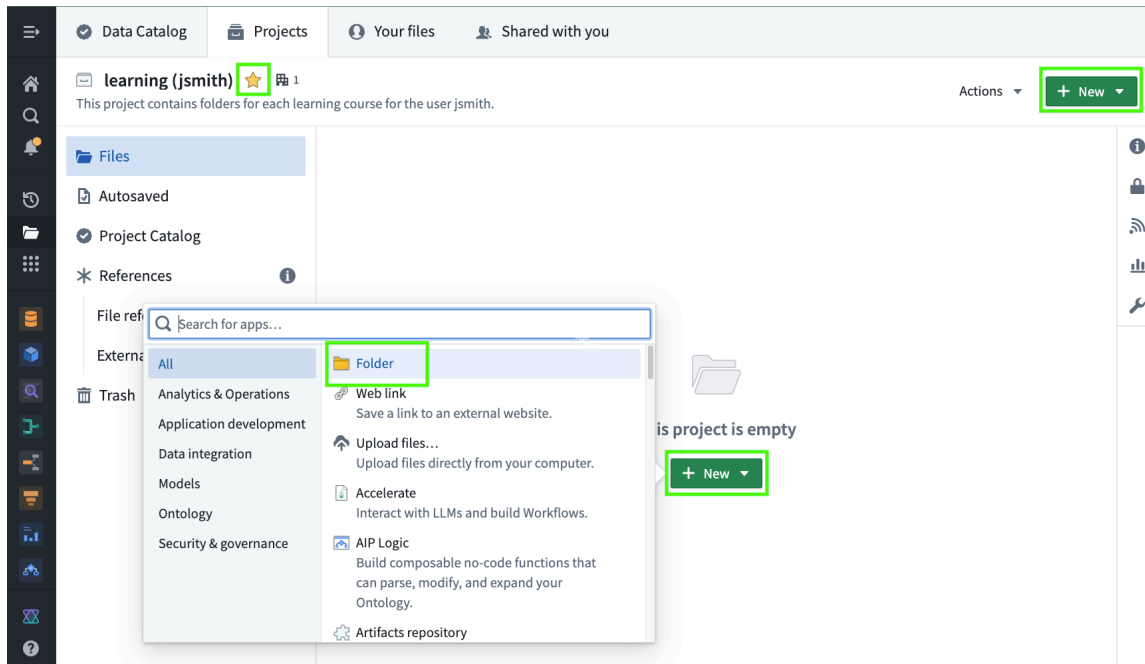
- [Create a project](#)

Create a Course-Specific Training Folder

Lesson 5 / 25 · [原文链接](#)

Step 1: Create your Folder

1. In the top left, click on the star to favorite your Project. This will allow you to find it quickly later on.
2. Click on **New**.
3. Click on **Folder**.



(Tip: Right-click on the image and click "Open Image in New Tab" to see the full-size version)

Step 2: Name your Folder

1. Name the new folder after the current learning course
 - For example: Deep Dive: Building your first Pipeline

Introduction

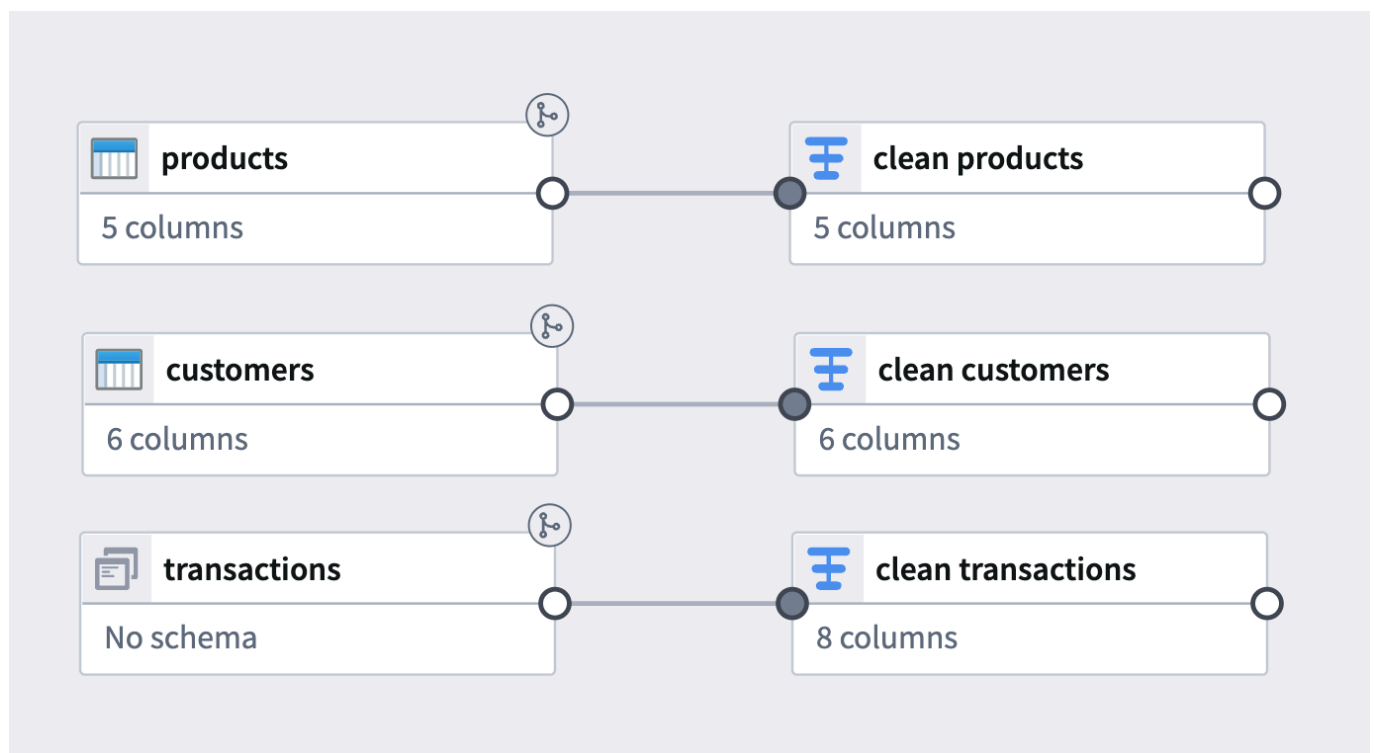
Lesson 6 / 25 · [原文链接](#)

In this module, you will focus on fundamental aspects of building a data pipeline in Foundry. The exercises in this section revolve around cleaning raw data, but the concepts and techniques discussed here are applicable beyond just data cleaning and preprocessing.

Building a data pipeline involves ensuring that the raw data coming directly from the source is consistent, accurate, and ready for analysis. This is crucial for downstream tasks, such as operational insights or further data integration. Foundry's Pipeline Builder simplifies these tasks by providing pre-built Spark modules that make it easy to address common data processing challenges, such as data type casting, file format conversion, string formatting, and many more.

In this module, you will walk through key data pipeline building concepts and understand how to leverage them using Pipeline Builder. For more information on Pipeline Builder, please refer to the [Foundry Documentation](#).

At the end of this module, you will have implemented a cleaning step for each of the uploaded raw datasets.



This course is being actively improved - we would love to hear your feedback at training-feedback@palantir.com

Download the Data Sources

Lesson 7 / 25 · [原文链接](#)

Now that you have your training location set up, you can upload the data you'll be working with. Below are the different files you'll need to download and then upload into Foundry. Click the names of the files below to download them to your device.

-  [products_raw.snappy.parquet](#): A raw `parquet` file containing information about the products sold at the store.
-  [customers_raw.csv](#): A raw `csv` file with basic information about the customers.
-  [transactions_raw.json](#): A raw `json` file containing transaction records, with details about the product, the customer, date, and quantity sold.

Important Downloading Tip: your browser might open the files themselves instead of downloading them (especially for the `json` file), use `Ctrl + S` or `cmd + S` to save the file.

If you are unable to download datasets from this site or are unable to manually upload datasets to your Foundry instance due to lack of permissions, you may be able to deploy the datasets to Foundry via the Marketplace application. Move on to the [\[Optional\] Deploy Datasets via Marketplace section](#) for instructions.

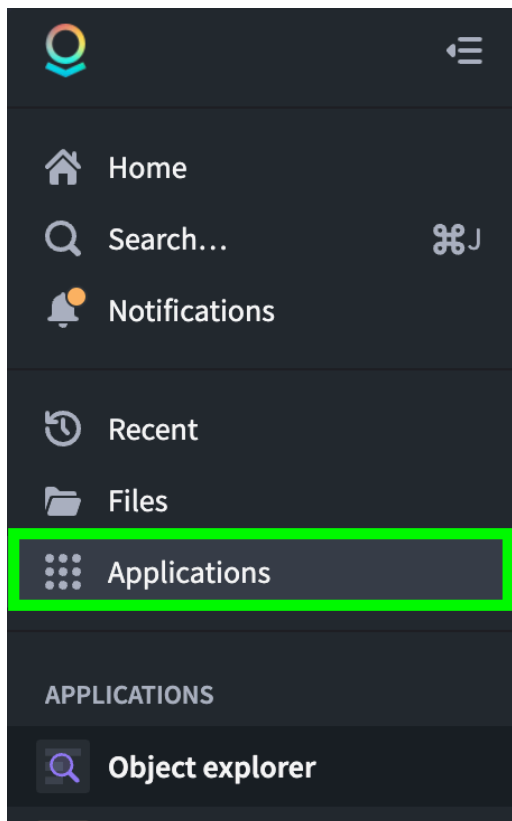
[Optional] Deploy Datasets via Marketplace

Lesson 8 / 25 · [原文链接](#)

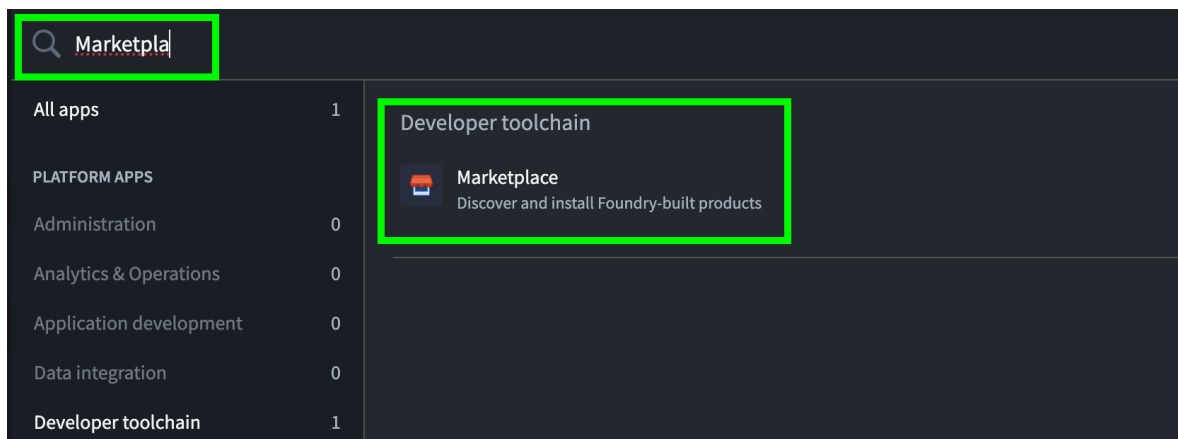
Only complete these steps if you are unable to download datasets from this site or are unable to upload datasets to your Foundry instance due to lack of permissions.

Step 1: Open Marketplace

1. Click on **Applications** on the left side of the screen

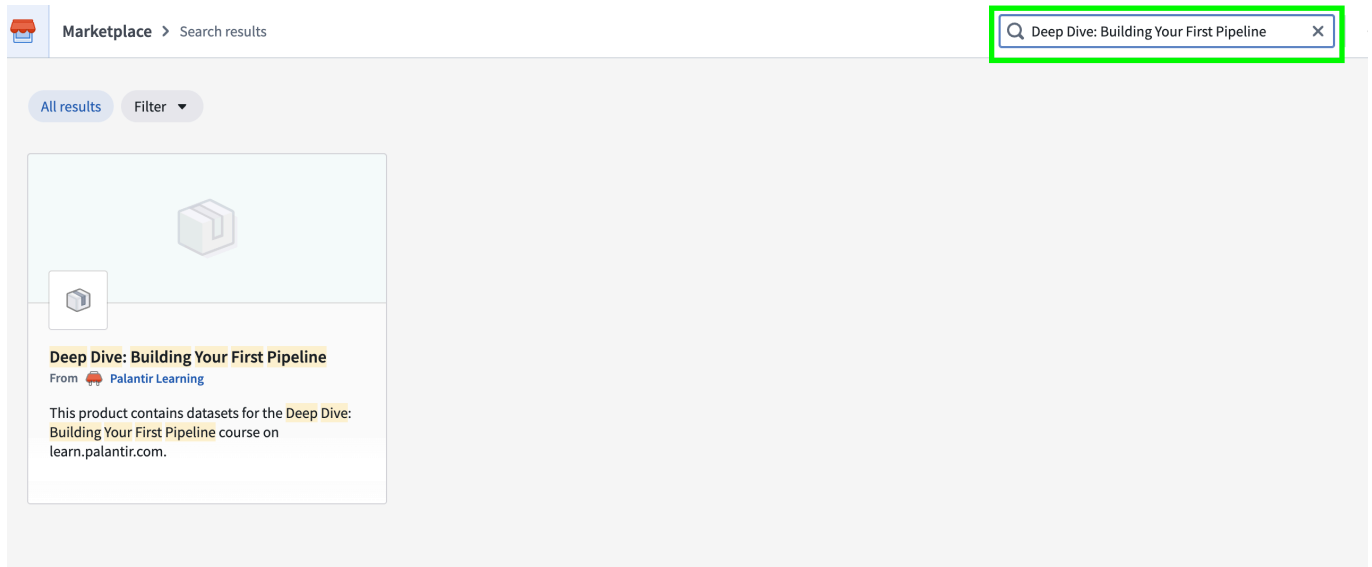


2. Search for `Marketplace` and open it.



Step 2: Open the Marketplace product

1. Search for the Deep Dive: Building Your First Pipeline product in the top search bar labeled **Search products...** (not the lower search bar labeled **Search stores...**)
2. Click into the corresponding product
3. Click through the prompts to install the datasets via Marketplace and save them in your folder for this course. Detailed instructions for installing a similar Marketplace package are listed in the [Deep Dive: Building Your First Application course](#).



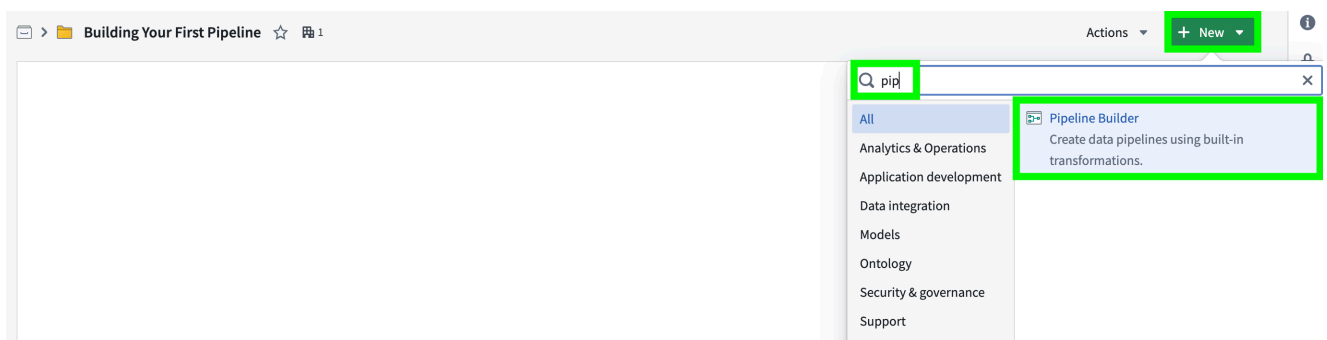
Create a New Pipeline

Lesson 9 / 25 · [原文链接](#)

In this step, you will create a new [batch pipeline](#) using Pipeline Builder, where you will add the raw data as input, and will clean the data to be later used for analysis and further data integration. The pipeline is the central place where you will be transforming your data.

Step 1: Add a new pipeline

1. In your working directory, create a new pipeline.



Step 2: Rename and select batch

1. Name your new pipeline "*Pipeline*"
2. Select the pipeline style as *Batch pipeline*
3. Select the batch compute as *Standard*
4. Click on *Create pipeline*

Create new pipeline

Get started on transforming your data with a new pipeline

Name and location

 pipeline

 Edit location

Deep Dive: Building Your First Pipeline

Configuration

**Batch pipeline**
Builds and transforms entire datasets on each deploy. For data that is ingested periodically.

**Streaming pipeline**
Transforms data continuously as new data is made available. For high frequency ingestion.

Select batch compute

**Standard**

Default

Build your pipelines using advanced expressions. Backed by Spark.

**Faster**

New

Speed up your pipelines and save compute. Backed by DataFusion.
[Learn More](#)

**External**

Beta

Build your pipelines using external compute platforms. [Learn More](#)

[Compare types](#)

 Back

Create pipeline

Step 3: Add the data for the pipeline

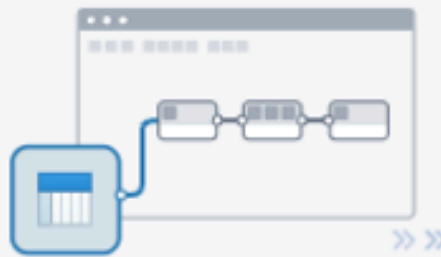
1. When opening Pipeline Builder, choose **Upload data from your computer**. If you deployed the datasets using marketplace, use the **Add Foundry data** option.
2. When the window opens, select the **products** (parquet) and **customers** (CSV) datasets from your device.

Note: We'll upload these two files together as the platform is able to infer the schema (column names) of these two data types. The import option for the json file will look slightly different in the next steps.

Welcome to Pipeline Builder

Get started by adding datasets, then define transform logic to derive target outputs.

 [Take a tour](#)



Add Foundry data

Recommended if you have already ingested data into Foundry.

Add data to Foundry

Import data from outside Foundry and start using it now

Upload from your computer

Recommended if you have sample data available locally.

Manually enter data

Recommended if you do not have data available to import.

3. In the following window, rename the datasets by clicking into the existing name. Name the parquet file `products` and the CSV file `customers`.
4. Select **Upload as individual structured datasets**.
5. Click **Upload**.

Upload files



Drop files here or [choose from your computer](#)

 customers.csv

10.23 KB 



products

5.69 KB 

☒ Upload as individual structured datasets (recommended)

Datasets are the most basic representation of tabular data. They can be used and transformed by many different applications. [Documentation](#) ➔

☐ Upload to a new media set

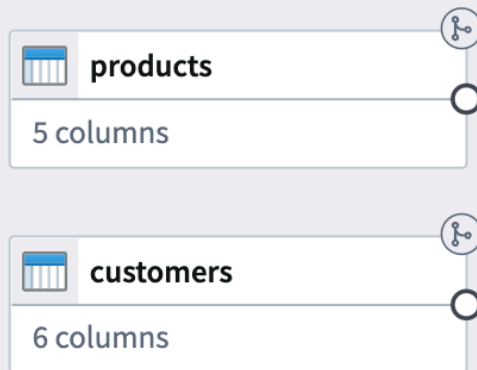
Media sets enable media-specific capabilities for media files (e.g. audio, imagery, video, and documents). [Documentation](#) ➔

☐ Upload to a new unstructured dataset

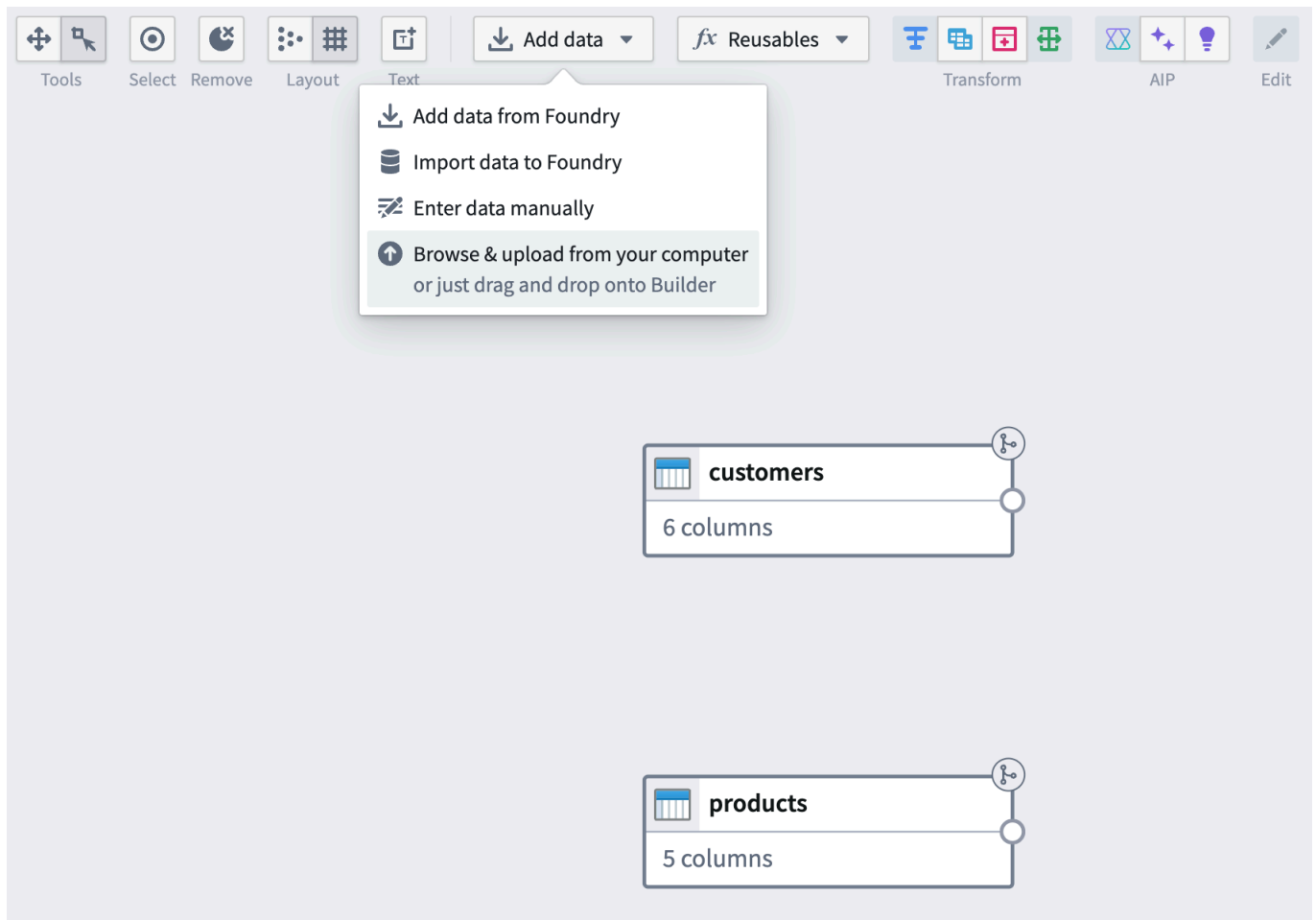
Unstructured datasets can store arbitrary files for processing and analysis. Structured data can be extracted from unstructured datasets using Pipeline Builder or Transforms. [Documentation](#) ➔

Upload

6. You should now see the products and customers datasets in your pipeline represented as rectangular nodes



7. Next, we need to upload the json file for **transactions**.
8. From the top left bar, select **Add Data** and **Browse & upload from your computer**



9. Find and select the json file for transactions on your device
10. Rename the file to `transactions`
11. Select **Upload to a new unstructured dataset**
12. Click **Upload**

 Upload files ×



Drop files here or [choose from your computer](#)

 instructor_7ak1fc9fmw3tjm70cm0drhgkg_public_17080791... 15.18 KB ×

☐ **Upload to a new media set (recommended)**

Media sets enable media-specific capabilities for media files (e.g. audio, imagery, video, and documents). [Documentation](#)

☒ **Upload to a new unstructured dataset**

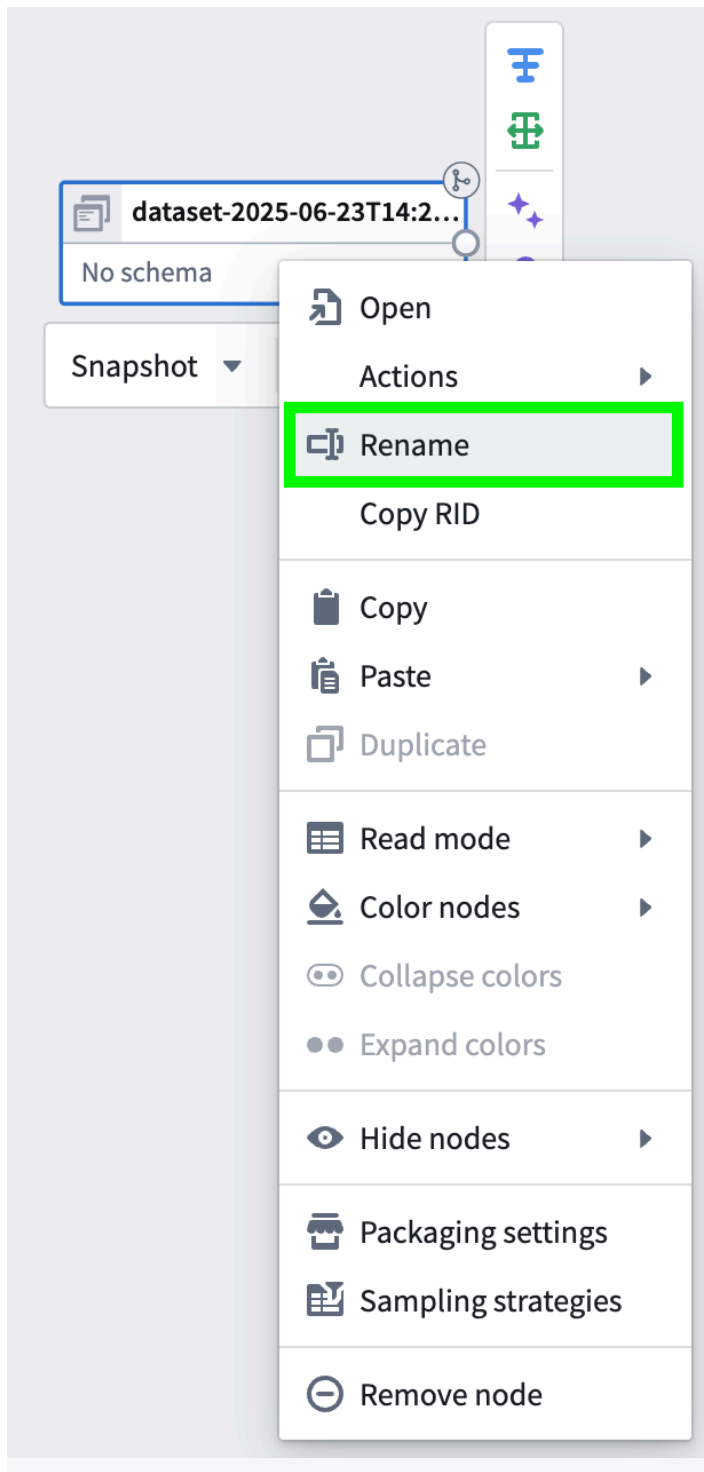
Unstructured datasets can store arbitrary files for processing and analysis. Structured data can be extracted from unstructured datasets using Pipeline Builder or Transforms. [Documentation](#)

Upload

Note that the 'transactions' name doesn't carry over to the name of the node representing the dataset in your pipeline as we need to complete some extra steps to extract the schema of the json file into a proper dataset. For now, we will manually rename the node for naming consistency.

13. Right / control-click on the node representing the transactions data

14. Click **Rename** and input `transactions`



15. You should now have three nodes on your screen for customers, products, and transactions

**transactions**

No schema

**products**

5 columns

**customers**

6 columns

Basic Transforms: Clean the Products Dataset

Lesson 10 / 25 · [原文链接](#)

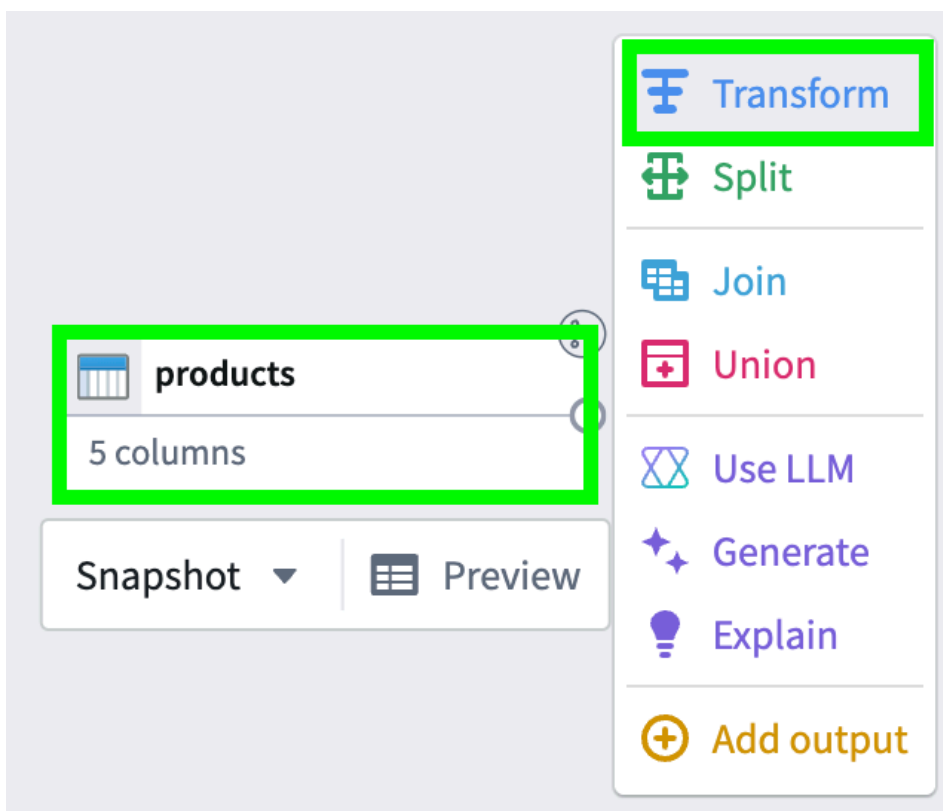
You are ready to start transforming your data right away. Typically, raw data is likely to present certain challenges, stemming either from the ingestion phase or directly from anomalies in the storage system of the source. The `products` dataset has some minor issues which can be addressed with pre-built Pipeline Builder modules in a cleaning transform. These out-of-the-box transforms make it easy to perform common data cleaning tasks quickly and efficiently.

In this section, you will solve the following issues:

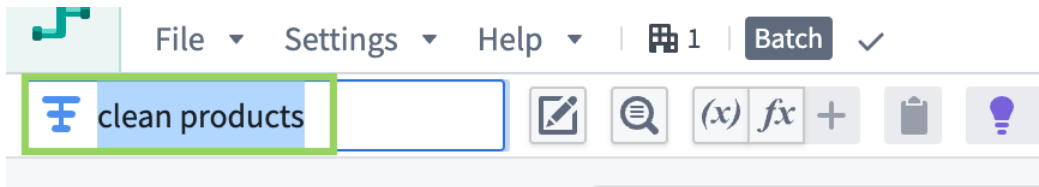
- The product name column has trailing whitespaces.
- The price column has been ingested as a string, but it should be a double data type.

Step 1: Create a new transform for raw `products` dataset

1. Select the **products** dataset and choose **Transform**.

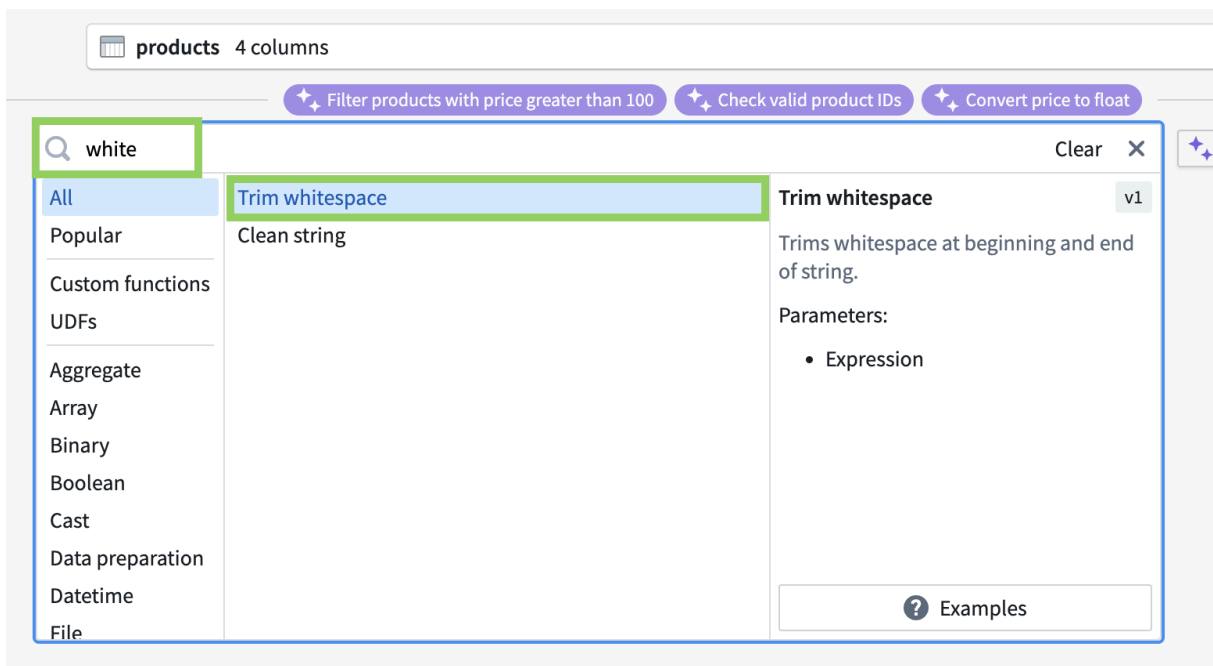


2. Give the transform a relevant name (rename the transform from the top left corner).

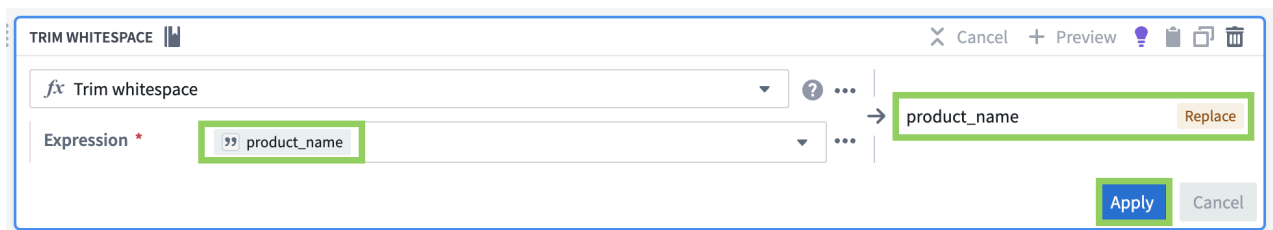


Step 2: Remove whitespaces in product name

1. Search for the **Trim whitespace** transformation.



2. Select column **product_name** and **Apply**.



Step 3: Cast price to double

1. Search for the **Cast** transform, select it, and cast the **price** column to **double**. (For accuracy, currency is often stored as integers in the smallest unit (e.g., cents) to avoid rounding errors. In this course, we use **double** for simplicity, as our values are small and precision issues are unlikely)

Trim whitespace → product_name

Edit Preview

CAST

Cancel Preview

fx Cast to Double

?

...

Expression *

price

...

price

Replace

Type *

Double

Applied

Cancel

Step 4: Apply all changes

1. Click **Apply all changes**.
2. Click **Close** to take you back to the page displaying dataset nodes.

Advanced Transforms: Clean the Customers Dataset

Lesson 11 / 25 · [原文链接](#)

In this section, you will analyze the shape of your data to identify specific cleaning and processing requirements. Datasets can vary in complexity, and understanding the structure and characteristics of your data is crucial for determining the appropriate cleaning steps.

[Preview](#) in Pipeline Builder is a great tool to identify aspects of your data that need to be cleaned or transformed. By utilizing this preview functionality, you can gain insights into potential issues or inconsistencies within your data, allowing you to make informed decisions about the necessary cleaning processes.

Additionally, you will walk through less straightforward transformations, to mimic real world examples whereby raw data can come in various shapes. This section will allow you to deepen your understanding of the Pipeline Builder transform boards, by constructing complex transformations and nesting the results.

Step 1: Explore customers dataset using Preview

1. Click on the **customers** dataset to open Preview.

The screenshot shows the Pipeline Builder interface. On the left, a sidebar lists datasets, with 'customers' (6 columns) highlighted. The main area displays a preview of the 'customers' dataset, showing 59 rows and 6 columns. The columns are: registration_date (Timestamp), registration_date_t... (String), email (String), customer_id (String), name (String), and address (String). The preview table contains 18 rows of data, including names like Margaret O'Kon, Jim Harber, Jodi Hyatt, Marty Lockman, Mrs. Regina Kautzer, Mr. Kristen Bins, Garrett Beahan, Bernadette Rolfson, Sabrina Schinner, Dr. Brett Monahan I, Sam Legros, Courtney Schneider, Trevor Hoppe, Terrell Lesch Jr., Michael Yundt, Olive Steuber, Lucia Wuckert, and Willard Predovic. A 'Calculate row count' button is visible in the bottom right corner of the preview table.

	registration_date	registration_date_t...	email	customer_id	name	address
1	2024-01-30T03:11:32.000Z	-05:00	margaret.o@example.com	f705437c-be19-45a8-8af9	Margaret O'Kon	{"city": "Yoshikof
2	2024-01-01T07:15:52.000Z	-05:00	jim.harber@example.com	0ae411be-4c04-46fd-a552	Jim Harber	{"city": "Daphneyt
3	2023-09-28T13:12:13.000Z	-04:00	jodi.hyatt@example.com	51c246fd-d4b8-4e62-97e9	Jodi Hyatt	{"city": "East Jen
4	2023-11-29T21:19:46.000Z	-05:00	marty.lockman@example.c	62470fcb-4b42-45d7-85f4	Marty Lockman	{"city": "North Em
5	2023-06-24T07:20:01.000Z	-04:00	mrs.kautzer@example.com	d638134a-5d98-4a53-84f6	Mrs. Regina Kautzer	{"city": "Jaunitab
6	2023-07-26T01:47:04.000Z	-04:00	mr.bins@example.com	ee1746a3-77f9-4688-86e8	Mr. Kristen Bins	{"city": "Brookhav
7	2024-01-09T09:23:00.000Z	-05:00	garrett.beahan@example.	e4fa2da8-c044-4468-bfee	Garrett Beahan	{"city": "Port Kav
8	2023-03-16T21:44:08.000Z	-04:00	bernadette.rolfson@exam	ac6ec763-0204-4634-99c5	Bernadette Rolfson	{"city": "New Nico
9	2023-10-12T14:27:59.000Z	-04:00	sabrina.schinner@exampl	9846accc-2a17-4aa0-867d	Sabrina Schinner	{"city": "Abilene"
10	2023-08-04T21:45:59.000Z	-04:00	dr.monahan@example.com	8a60c199-6d25-4c79-9dfa	Dr. Brett Monahan I	{"city": "North Gi
11	2024-01-03T01:59:35.000Z	-05:00	sam.legros@example.com	bc9b611a-0344-4d37-8d60	Sam Legros	{"city": "Daly Cit
12	2023-03-14T11:23:27.000Z	-04:00	courtney.schneider@exam	1475707f-c65e-4d07-a4de	Courtney Schneider	{"city": "South Ja
13	2023-09-14T14:19:02.000Z	-04:00	trevor.hoppe@example.co	4eb73204-c1ee-47df-a651	Trevor Hoppe	{"city": "New Mael
14	2023-03-29T22:26:41.000Z	-04:00	terrell.lesch@example.c	f9f95fe6-517a-4964-9f96	Terrell Lesch Jr.	{"city": "East Row
15	2023-04-11T01:44:05.000Z	-04:00	michael.yundt@example.c	cdc59ba3-647d-41f6-8f0d	Michael Yundt	{"city": "Jeromysi
16	2023-07-05T20:42:24.000Z	-04:00	olive.steuber@example.c	961c0949-5aeb-421a-94d6	Olive Steuber	{"city": "Jeromysi
17	2023-10-28T06:13:43.000Z	-04:00	lucia.wuckert@example.c	9e6e8df5-dae4-4b2a-83c3	Lucia Wuckert	{"city": "Fort Kar
18	2023-11-11T02:00:41.000Z	-05:00	willard.predovic@exampl	2350121e-13d3-44da-899a	Willard Predovic	{"city": "Fort Kar

2. View stats on the **customer_id** column.

	Showing 59 rows	6 columns	Search
--	-----------------	-----------	--------

customer_id	name
String	String
f705437c-be19-45a8-8a1	
0ae411be-4c04-46fd-a55	
51c246fd-d4b8-4e62-97e	
62470fcb-4b42-45d7-85f	
d638134a-5d98-4a53-84f	
ee1746a3-77f9-4688-86e	
e4fa2da8-c044-4468-bfe	
ac6ec763-0204-4634-99c	
9846accb-2a17-4aa0-867	
8a60c199-6d25-4c79-9d1	
bc9b611a-0344-4d37-8d6	
1475707f-c65e-4d07-a4c	
4eb73204-c1ee-47df-a65	
f9f95fe6-517a-4964-9f9	
cdc59ba3-647d-41f6-8fc	
961c0949-5aeb-421a-94c	
9e6e8df5-dae4-4b2a-83c	

Pin column
Mark as policy column
Encrypt column
Filter
Sort ascending
Sort descending
View stats
View cell content
Copy column name
Expand

- Notice the values of some of the primary key are not unique, meaning that there are duplicates in the data.

customers	Showing all 59 rows	6 columns	Search columns...
-----------	---------------------	-----------	-------------------

customer_id	String	59 rows
Normal	59	
Null	0	
Empty	0	
Whitespace	0	
Needs trim	0	
Numeric	0	
Non-alpha	0	
Uppercase	0	
Lowercase	59	
Mixed case	0	
Distinct	50	

LENGTH	1	by inc. value	VALUE	50	by desc. count	Filter...
36	59		d638134a-5d98-4a53-84f6-30bc1d1909eb	2		
			415aa0fe-2fa3-413e-898b-699cb16228cd	2		
			8a60c199-6d25-4c79-9dfa-22ce59196e3e	2		
			d204bc7b-7469-4ef8-86e0-dabb4a88d160	2		
			388054af-26f1-4a65-8511-cd4cec3f5c1b	2		
			2350121e-13d3-44da-899a-08090029100a	2		
			52134279-d400-499b-9408-6ead049d14eb	2		
			9e6e8df5-dae4-4b2a-83c3-06b9b6cf574a	2		
			c5b5f385-b615-49c7-afe3-d444d8a56c5f	2		
			18113ded-1226-4cab-8ded-e9ce30116b4b	1		
			e5a1bb03-a103-4e99-9c4b-5d1cf6650410	1		

- Filter to those duplicated values to decide on the deduplication strategy. To do this, select the required rows (you can click one at a time or use shift + click to highlight multiple rows at once) and then click *Keep Values*.

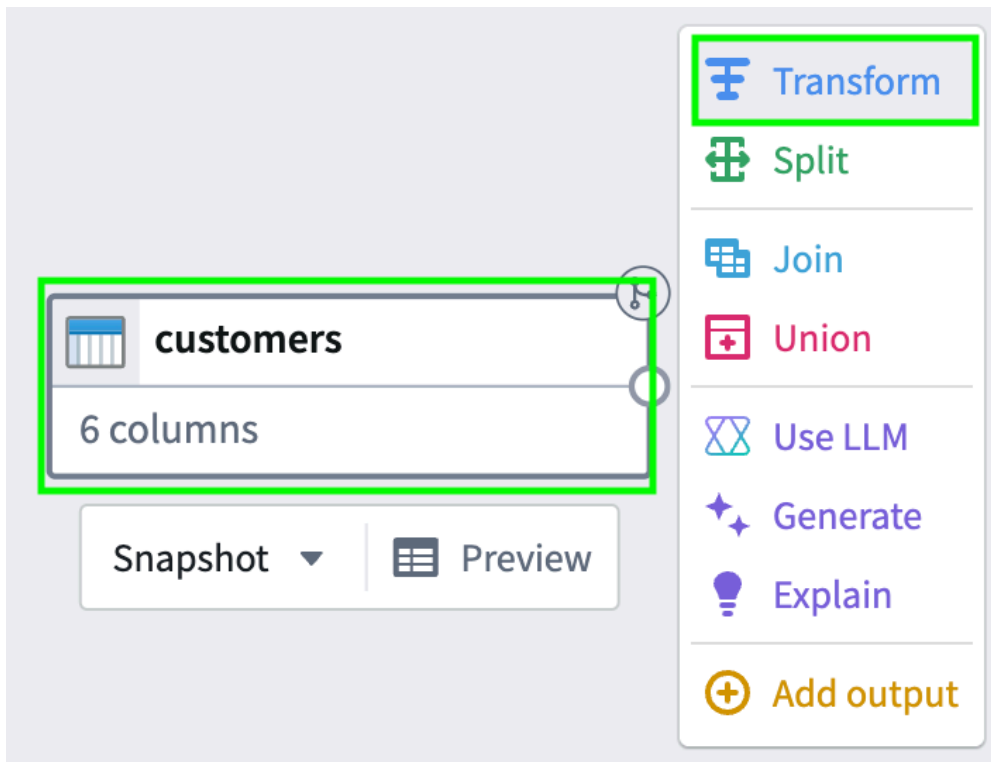
customer_id	String	59 rows	Keep Values	Remove Values	
Normal	59	LENGTH 1	by inc. value	VALUE 50	by desc. count
Null	0	36	59		
Empty	0				
Whitespace	0				
Needs trim	0				
Numeric	0				
Non-alpha	0				
Uppercase	0				
Lowercase	59				
Mixed case	0				
Distinct	50				

5. Click the X in the top right to go back to the preview of the dataset to see what values remain after filtering

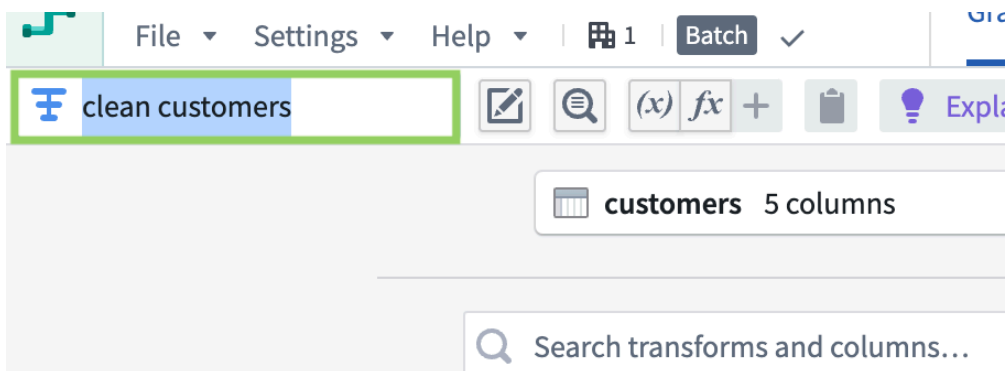
customers				Showing 18 rows	6 columns	Search columns...
No input sampling	customer_id: d638134a-5d98-4a53-8...					
	registration_date Timestamp	registration_date_t... String	email String	customer_id String	name String	address String
1	2023-06-24T07:20:01.000Z	-04:00	mrs.kautzer@example.com	d638134a-5d98-4a53-84f6	Mrs. Regina Kautzer	{"city": "Jaunitabu
2	2023-08-04T21:45:59.000Z	-04:00	dr.monahan@example.com	8a60c199-6d25-4c79-9dfa	Dr. Brett Monahan I	{"city": "North Gil
3	2023-10-28T06:13:43.000Z	-04:00	lucia.wuckert@example.c	9e6e8df5-dae4-4b2a-83c3	Lucia Wuckert	{"city": "Onaland",
4	2023-11-11T02:00:41.000Z	-05:00	willard.predovic@exampl	2350121e-13d3-44da-899a	Willard Predovic	{"city": "Fort Karl
5	2023-08-12T10:45:44.000Z	-04:00	edgar.green@example.com	d204bc7b-7469-4ef8-86e0	Edgar Green	{"city": "Parker",
6	2023-09-16T04:29:22.000Z	-04:00	darrin.mueller@example.	52134279-d400-499b-9408	Darrin Mueller	{"city": "Corwinbur
7	2023-08-05T02:49:59.000Z	-04:00	chester.maggio@example.	415aa0fe-2fa3-413e-898b	Chester Maggio	{"city": "Yesseniat
8	2023-05-06T05:40:04.000Z	-04:00	sara.dickinson@example.	388054af-26f1-4a65-8511	Sara Dickinson	{"city": "Lacey", "
9	2023-12-16T08:44:24.000Z	-05:00	jim.mccullough@example.	c5b5f385-b615-49c7-afe3	Jim McCullough	{"city": "Dickinson
10	2023-06-24T07:20:01.000Z	-04:00	null	d638134a-5d98-4a53-84f6	null	null
11	2023-08-04T21:45:59.000Z	-04:00	null	8a60c199-6d25-4c79-9dfa	null	null
12	2023-10-28T06:13:43.000Z	-04:00	null	9e6e8df5-dae4-4b2a-83c3	null	null
13	2023-11-11T02:00:41.000Z	-05:00	null	2350121e-13d3-44da-899a	null	null
14	2023-08-12T10:45:44.000Z	-04:00	null	d204bc7b-7469-4ef8-86e0	null	null
15	2023-09-16T04:29:22.000Z	-04:00	null	52134279-d400-499b-9408	null	null
16	2023-08-05T02:49:59.000Z	-04:00	null	415aa0fe-2fa3-413e-898b	null	null
17	2023-05-06T05:40:04.000Z	-04:00	null	388054af-26f1-4a65-8511	null	null
18	2023-12-16T08:44:24.000Z	-05:00	null	c5b5f385-b615-49c7-afe3	null	null

Step 2: Drop duplicates

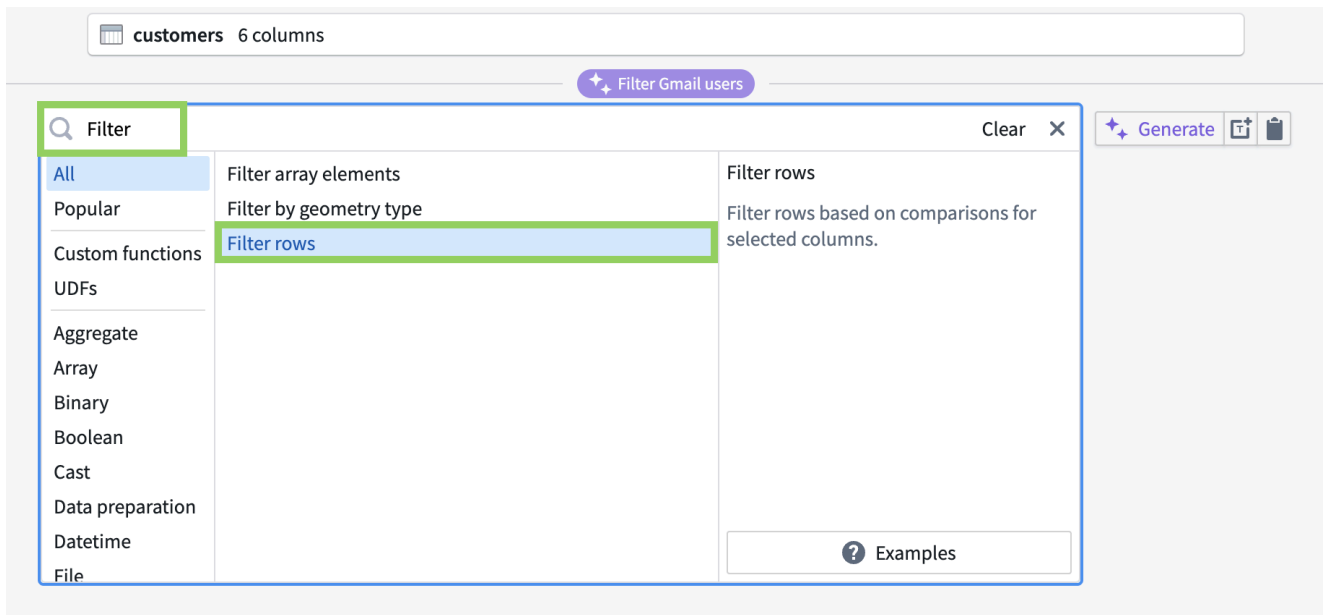
1. Similarly to the previous step, create a new transform to clean the raw **customers** dataset.



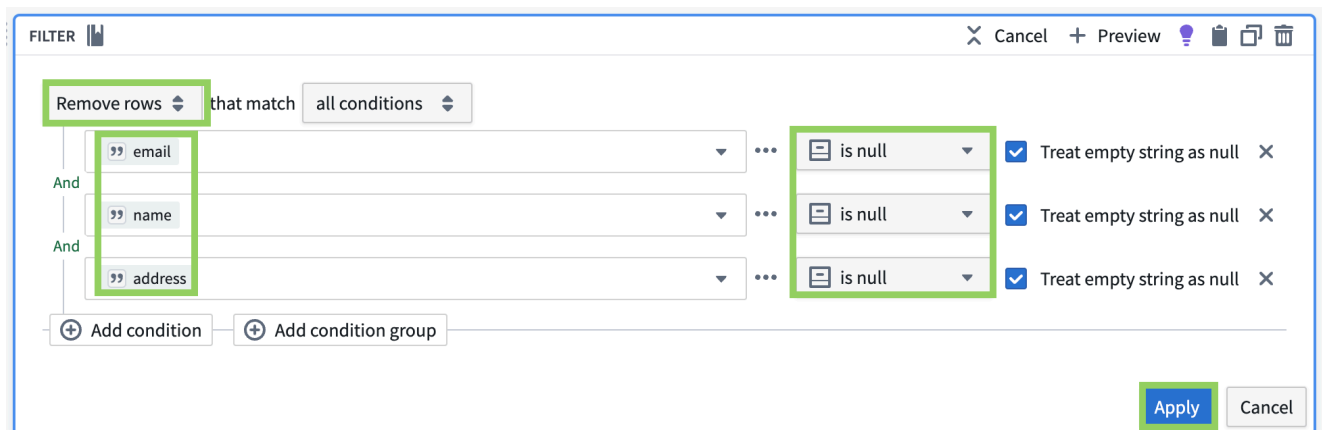
2. Give the transform a relevant name.



3. Search for the `Filter rows` transform.



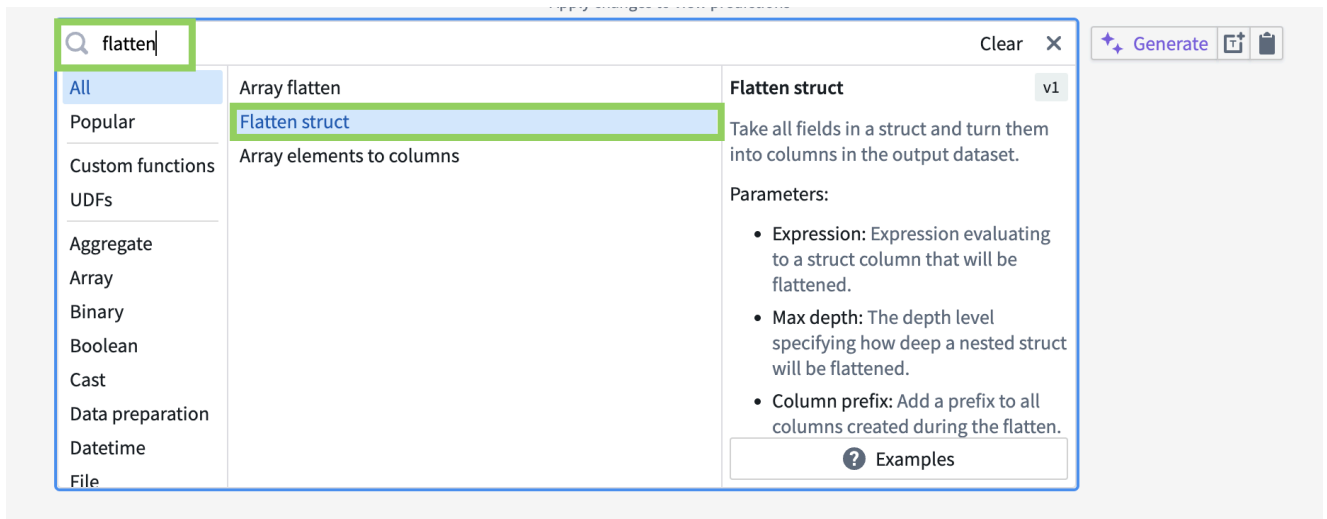
4. Configure the filter to Remove rows where **email**, **name** and **address** are **null**. Click **Apply**.



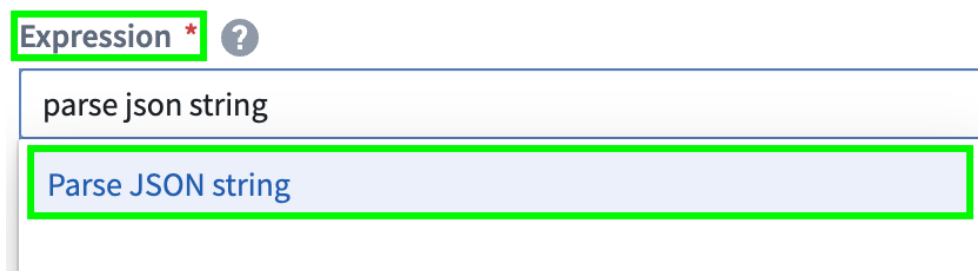
Step 3: Clean the address field

Notice that the **address** field is in JSON format and doesn't convey the information easily for the end user. We can use Pipeline Builder transforms to create a clean string with the **street** + **city** + **state** format. You will create a nested Transform, to flatten the struct defined by the address JSON.

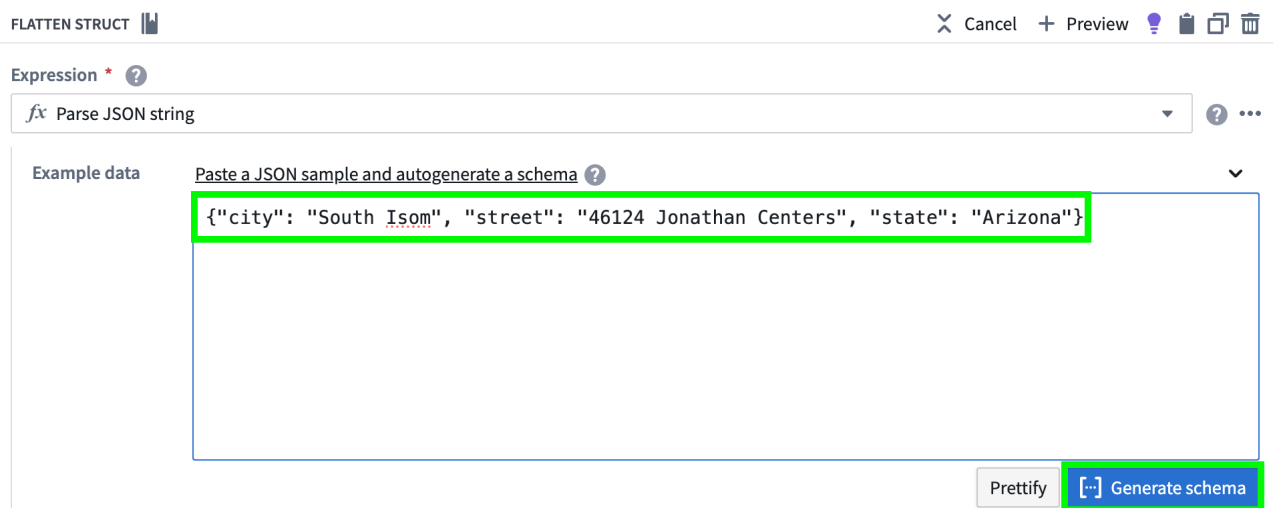
1. Search for the **Flatten struct** transform.



2. The **Flatten struct** transform requires a struct type input (a type of data format), so you need to parse the JSON string to a struct.
3. Select the **Parse JSON as Struct** expression.



4. Paste in the below example address to infer the schema.
 - `{"city": "South Isom", "street": "46124 Jonathan Centers", "state": "Arizona"}`



5. The schema should be inferred as below. Under **JSON** select the *address* property, as that contains our JSON string

Expression * ?

fx Parse JSON string

Example data [Paste a JSON sample and autogenerate a schema](#) ?

JSON schema [Paste a JSON schema to use](#)

JSON * ?

» address

Schema * ?

Struct

Name	Type
city	String
street	String
state	String

+ Add field

6. Your final Flatten struct transform should look similar to the below. You may need to change the underscore in **Separator** to a space:

FLATTEN STRUCT
Cancel
Preview

Expression * ?

fx
Parse JSON string

Example data
Paste a JSON sample and autogenerate a schema ?

JSON schema
Paste a JSON schema to use

JSON * ?

" address

Schema * ?

Struct

Name	Type
city	String
street	String
state	String

Add field

Output mode * ?

Simple

Max depth * ?

1

Column prefix * ?

Empty string

Separator * ?

1 space

Apply
Cancel

- This board creates three columns, **city**, **state**, and **street**. Concatenating them into a new **address** column will result in a readable string column. For now, click **Apply**.
- Search for the Concatenate Strings transform and select the **city**, **state**, and **street** columns. Add **,** as the Separator.
- Name the output column **address**.
- Click **Apply**.

Work with Different File Types: Preprocess and Clean the Transactions Dataset

Lesson 12 / 25 · [原文链接](#)

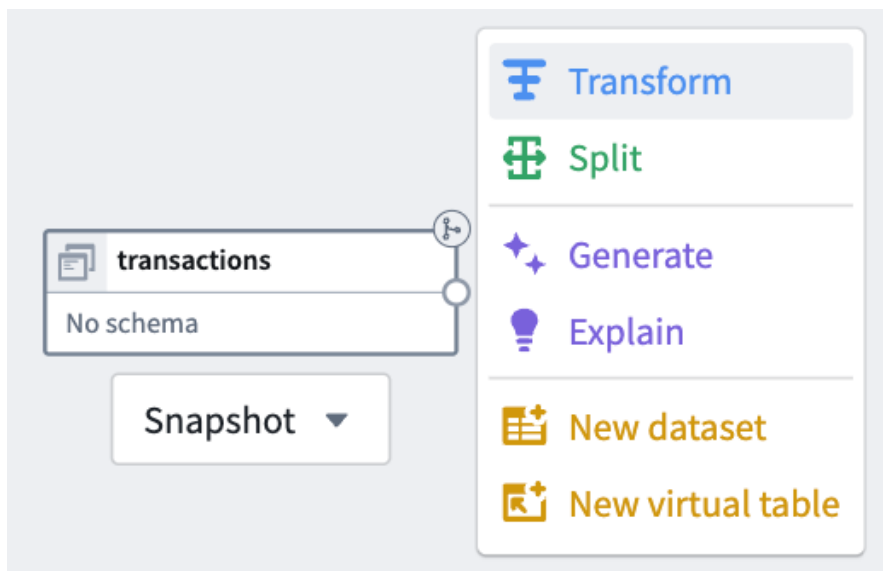
Datasets in Foundry are collections of files, generally Parquet. However, raw data may arrive in different formats, and your pipeline needs to be able to handle these formats and transform them into adequate Foundry datasets. For more information on the internal structure of Foundry datasets, please refer to the [Foundry Datasets Documentation](#).

Throughout this section, you will walk through an example of handling a raw dataset in JSON format using readily available Pipeline Builder transforms. This will provide you with a practical understanding of how to manage various file formats in your pipeline.

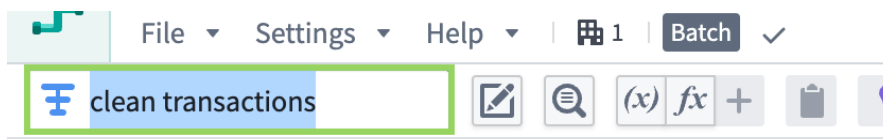
Additionally, you will be introduced to Foundry's AI assistant, which, if available on your Foundry installation, can be a valuable resource for simplifying the processing and transformation. By leveraging the AI assistant's capabilities, you can streamline the handling of various formats, further enhancing the efficiency and performance of your pipeline.

Step 1: Create a new transform for the `transactions` dataset

1. Similarly to the previous datasets, create a new transform.

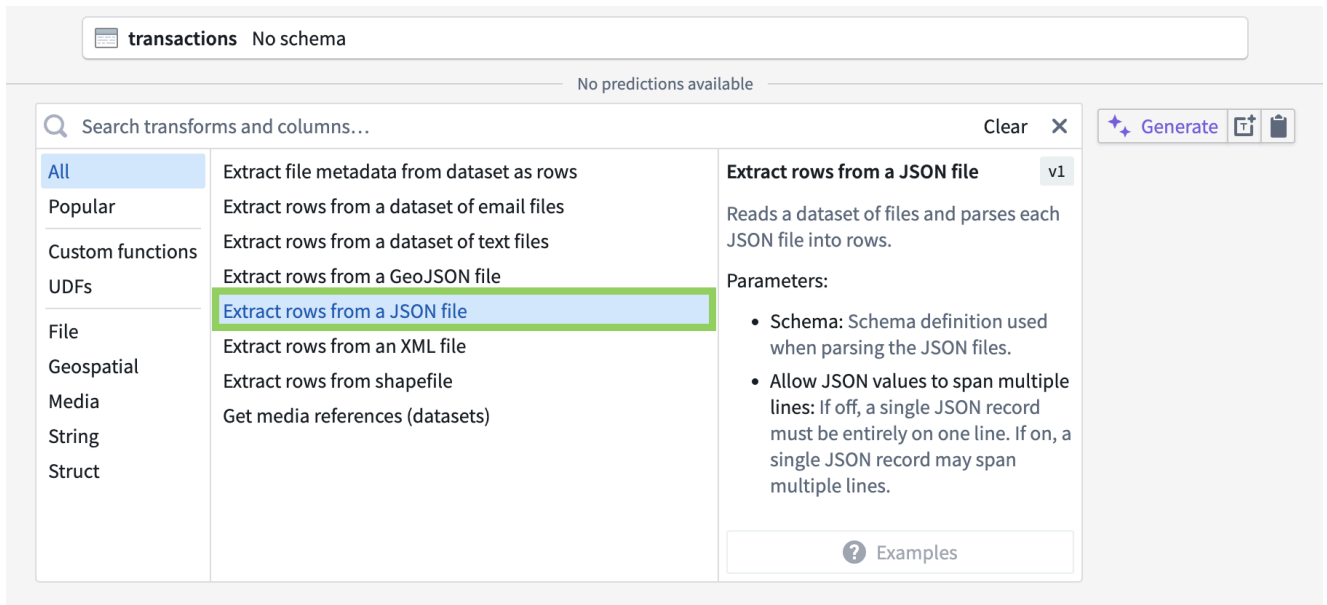


2. Give it a relevant name.



Step 2: Transform from JSON to dataset

1. Select the **Extract rows from a JSON file** transform



2. Paste in a sample JSON to auto-generate the schema. You can get one by opening the transactions file locally.

- For example,

```
{"transaction_id":"02a10def-53bc-4eb8-af6c-452e431d2511","transaction_date":"Thu Jan 25 2024 07:21:01 GMT+0000 (Greenwich Mean Time)","units":"610.00","customer_id":"f9f95fe6-517a-4964-9f96-e17030cfbf3c","variation_id":"53a47d0e-67a4-4241-835e-293e0c65d5fd","product_id":"77d64cec-d43b-4115-ab57-c646d01cac8d"}
```

Example data

Paste a JSON sample and autogenerate a schema ?

```
{"transaction_id":"02a10def-53bc-4eb8-af6c-452e431d2511","transaction_date":"Thu Jan 25 2024 07:21:01 GMT+0000 (Greenwich Mean Time)","units":"610.00","customer_id":"f9f95fe6-517a-4964-9f96-e17030cfbf3c","variation_id":"53a47d0e-67a4-4241-835e-293e0c65d5fd","product_id":"77d64cec-d43b-4115-ab57-c646d01cac8d"}
```

Prettify

Generate schema

JSON schema

Paste a JSON schema to use

Schema *

Struct

Name

transaction_id

Type

String

Name

transaction_date

Type

String

Name

units

Type

String

Name

customer_id

Type

String

Name

variation_id

Type

String

Name

product_id

Type

String

Add field

☐ Allow JSON values to span multiple lines ?

Apply

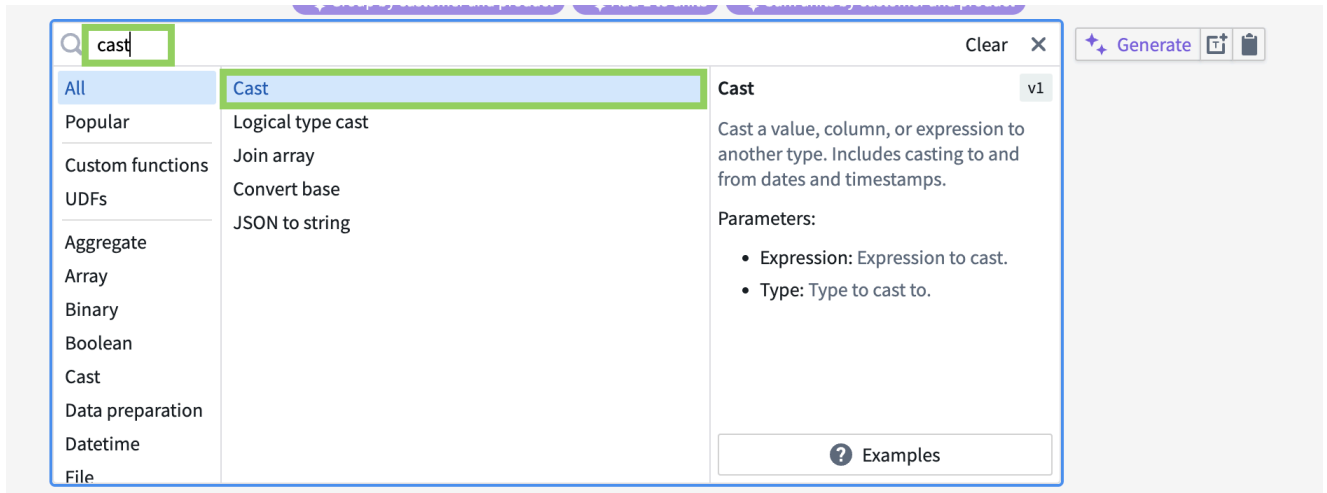
Cancel

3. Toggle the “Allow JSON values to span multiple lines” option to False (because we have one JSON file per row).

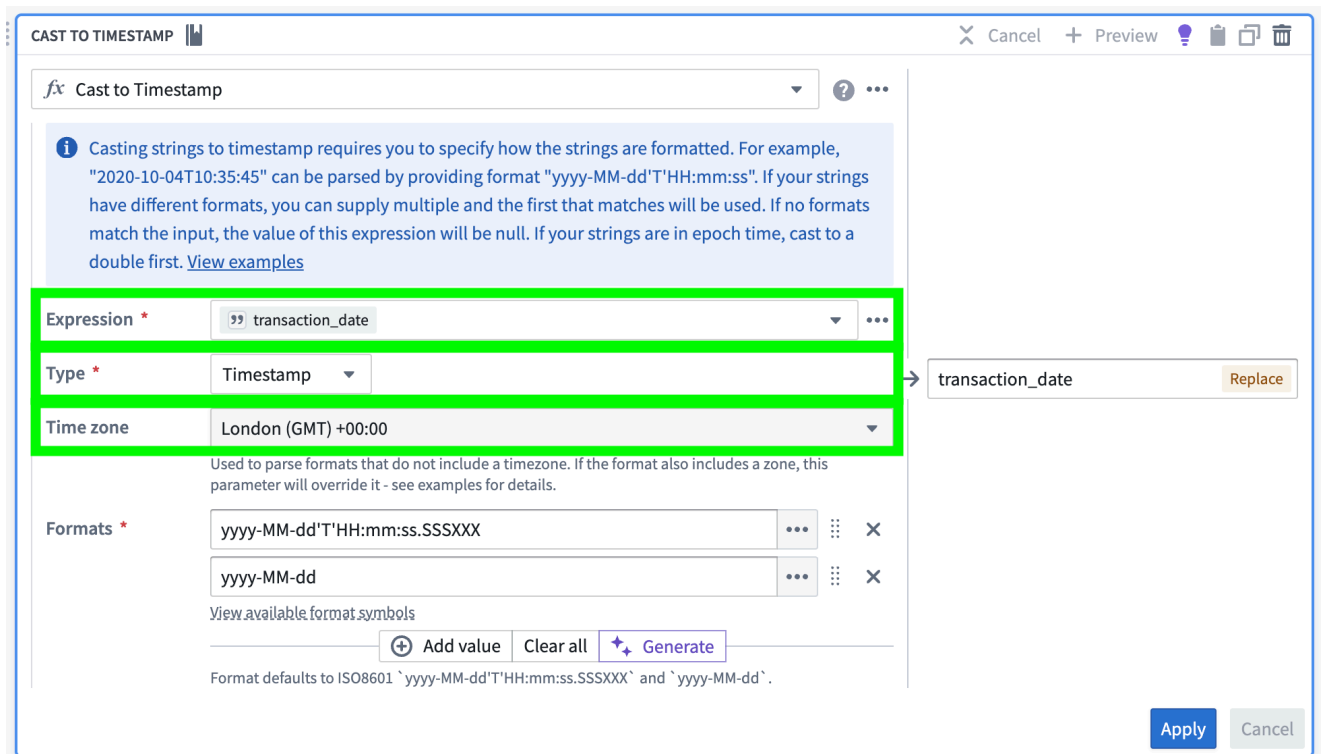
☐ Allow JSON values to span multiple lines ?

Step 3: Cast transaction_date to Timestamp

1. Search for the **Cast** transform.



2. Select 'transaction_date' as the **Expression**.
3. Set the **Type** to 'Timestamp'
4. Set the **Timezone** to 'London GMT'



5. Use the AI Assistant (Generate button) to infer the correct Timestamp format from an example string from the dataset. You can use `Thu Jan 25 2024 07:21:01 GMT+0000 (Greenwich Mean Time)` as an example value, then click Generate
 - If AI Assistant is not available on your Foundry instance, just paste in the format: `EEE MMM dd yyyy HH:mm:ss 'GMT'Z (zzzz)`.

Time zone London (GMT)

Used to parse for parameter will override it - see examples for details.

Formats *

yyyy-MM-dd' Example value, eg: '2000-01-01' ✓

yyyy-MM-dd' Examples: Thu Jan 25 2024 07:21:01 GMT+0000 (Greenwich Mean Time) ✕

View available format symbols

✕ Add value Clear all ✕ Generate

Format defaults to ISO8601 `yyyy-MM-dd'T'HH:mm:ss.SSSXXX` and `yyyy-MM-dd`.

Apply Cancel

6. Then click remove the other formats and click **Apply**.

CAST TO TIMESTAMP

fx Cast to Timestamp

i Casting strings to timestamp requires you to specify how the strings are formatted. For example, "2020-10-04T10:35:45" can be parsed by providing format "yyyy-MM-dd'T'HH:mm:ss". If your strings have different formats, you can supply multiple and the first that matches will be used. If no formats match the input, the value of this expression will be null. If your strings are in epoch time, cast to a double first. [View examples](#)

Expression * transaction_date

Type * Timestamp

Time zone London (GMT) +00:00

Used to parse formats that do not include a timezone. If the format also includes a zone, this parameter will override it - see examples for details.

Formats *

yyyy-MM-dd'T'HH:mm:ss.SSSXXX ... ✕

yyyy-MM-dd ... ✕

EEE MMM dd yyyy HH:mm:ss 'GMT'Z ' ('zzzz')' ... ✕

View available format symbols

✕ Add value Clear all ✕ Generate

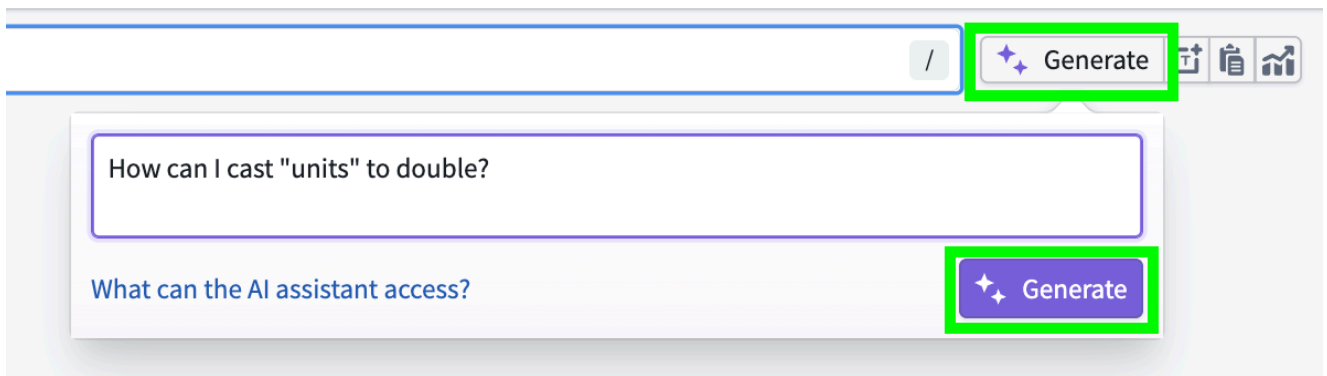
Format defaults to ISO8601 `yyyy-MM-dd'T'HH:mm:ss.SSSXXX` and `yyyy-MM-dd`.

transaction_date Replace

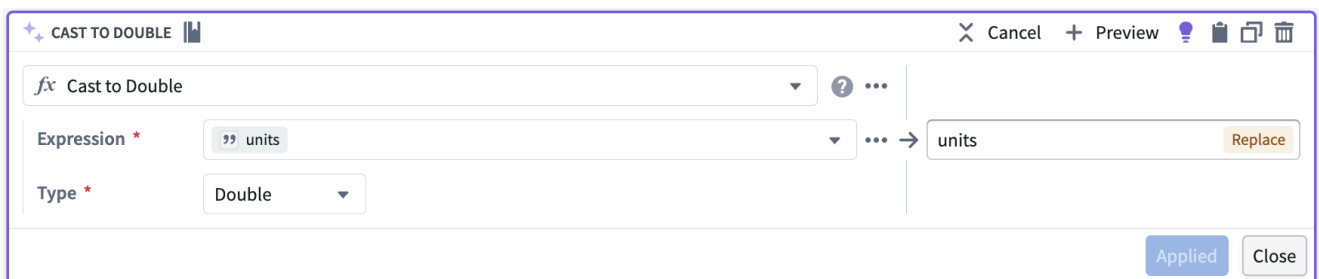
Apply Cancel

Step 4: Cast units to double

1. If available on your Foundry instance, you can use the AI Assistant again to generate a transform casting the units column to a double data type. If you don't have the AI Assistant available, simply select the **Cast** transformation for a new board.



2. The AI assistant should generate the board similar to the one below. There may be missing pieces in it, so you might need to adjust it.



Step 5: Drop unused columns

The conversion from JSON generated two additional columns, **_error** and **file**. These are not needed for this specific pipeline, so it's safe to drop them. When processing many files, these could be valuable to catch any formatting errors in the raw files.

1. Search for the Drop Columns transform.
2. Select **_error** and **file**.
3. Click **Apply**.
4. Make sure you have applied all changes and click **Close** to return to the main screen with the dataset nodes.

DROP COLUMNS 

 Cancel

 Preview



Columns to drop * 

`”_error` 

`”_file` 

Search for columns...

Apply

Cancel

Introduction

Lesson 13 / 25 · [原文链接](#)

In this module, you will build upon the clean data generated previously, in order to derive operationally valuable datasets, focusing on generating a Customer Lifetime Value (CLV) dataset. The cleaned data in your pipeline is a valuable foundation for key metrics and analyses, and while it is essential, it becomes even more valuable when integrated to break down data silos and provide comprehensive insights.

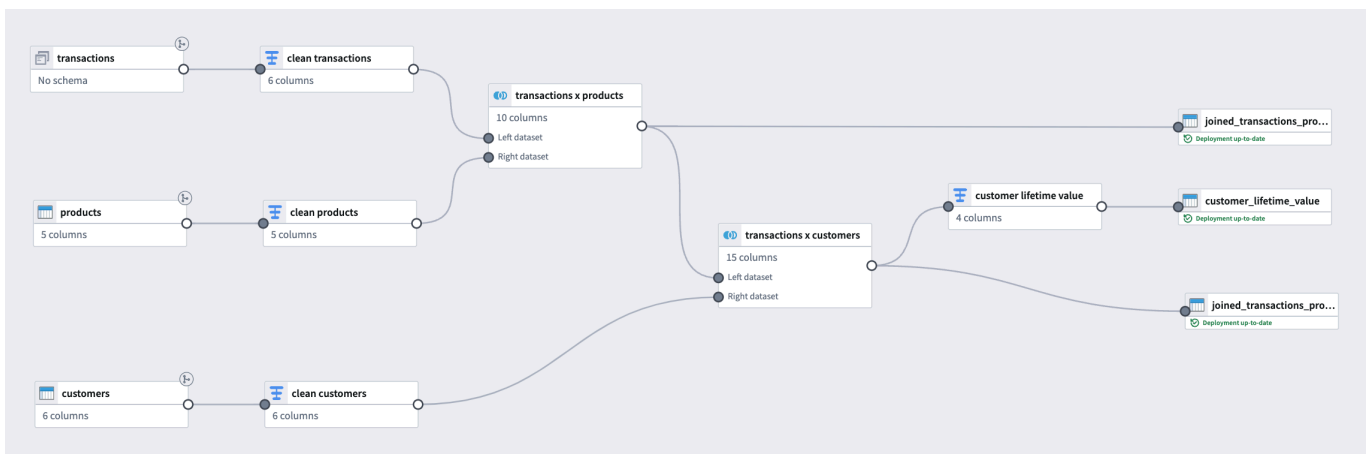
Throughout this section, you will gain hands-on experience with critical data processing techniques, such as joins and aggregations. Additionally, you will learn how to interpret the results of your pipeline to ensure that it delivers meaningful and accurate information.

In this module, you will explore the following concepts:

- **Joining and aggregation:** Learn how to combine and summarize datasets, while being mindful of potential pitfalls such as duplicates, incorrect join conditions, and the computational expense of joins.
- **Materializing outputs:** Understand the significance of outputs and why they matter.
- **Validating the output of the join:** Learn the importance of validating your joined dataset, as joins can sometimes introduce duplicates or errors if not executed correctly.

By the end of this section, you will have gained valuable insights into advanced pipeline building concepts and techniques. These skills will enable you to create more sophisticated, reliable, and efficient pipelines that effectively transform raw data into actionable intelligence.

At the end of this section, you will have created a pipeline similar to the one below:



This course is being actively improved - we would love to hear your feedback at training-feedback@palantir.com

Simple Joins: Join Transactions and Products

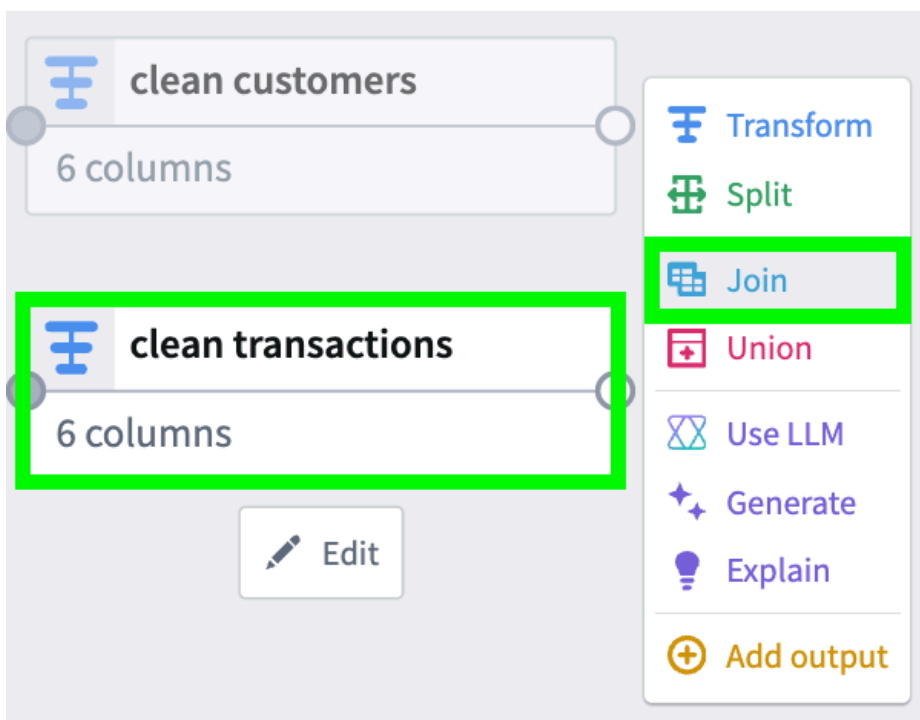
Lesson 14 / 25 · [原文链接](#)

To derive operationally valuable insights, it is often necessary to join different datasets, such as combining transactional data with product pricing, to paint a comprehensive picture of customer value and behavior. A Customer Lifetime Value dataset is one such example, showcasing the total revenue brought in by each customer since their registration.

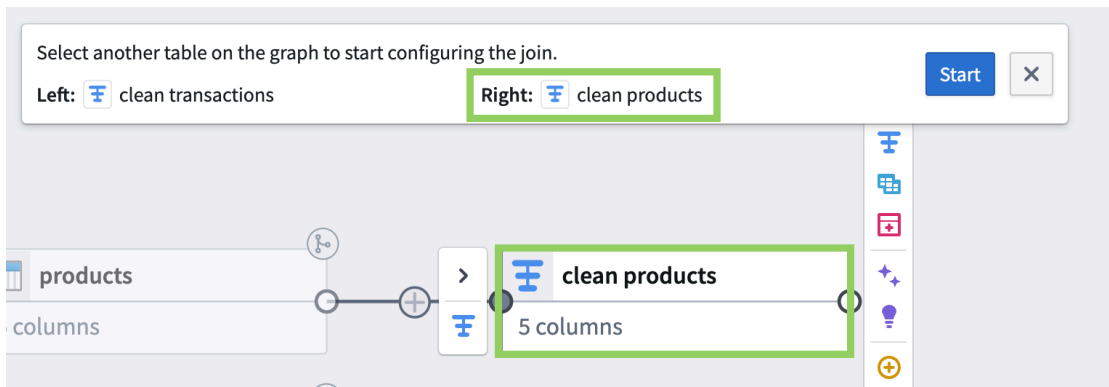
The `transactions` dataset holds information about all the purchases of all the customers, and the `products` dataset holds information about the price of each product. Joining those two will give a simple calculation for the revenue brought in by each transaction.

Step 1: Join clean transactions and clean products

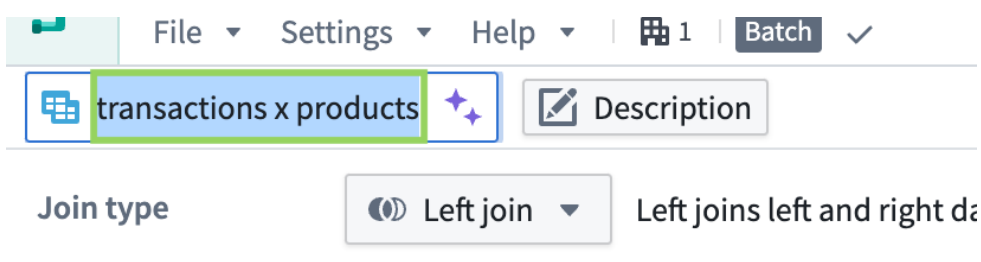
1. Select the **clean transactions** node and choose the **Join** option on the right hand side.



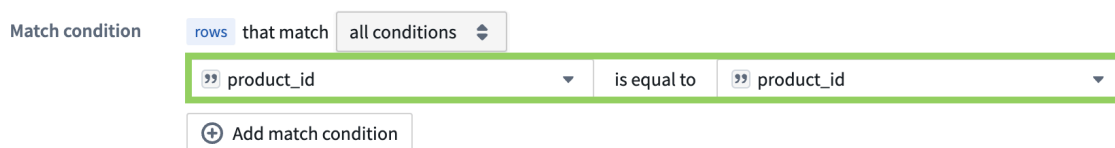
2. Choose the **clean products** node to join with. Click **Start**.



3. Give the join a relevant name.



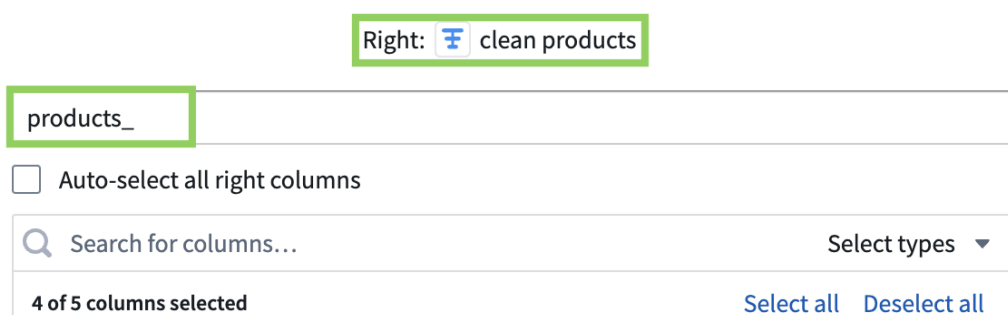
4. Set **product_id is equal to product_id** as the Join Condition.



Step 3: Prefix the product columns with products_

After we join the datasets, the resulting single dataset may have repeated column names. We need to add a prefix to some of the columns to distinguish which dataset they originated from.

1. On the right side of the screen, input `products_` into the prefix input bar.



Step 4: Verify Updates

1. Ensure you have the same columns selected in your join as in the screenshot below.
2. Click **Apply**
3. Click **Close** or **Back to graph**.

transactions x products

Description

AppliedBack to graph

Join type

Left joinLeft joins left and right dataset inputs together.

Input tables

clean transactionsSwapclean products

Match condition

rows that matchall conditions

product_id is equal to product_id

Add match condition

BasicAdvanced

Select columns

Left: clean transactions

Right: clean products

☒ Auto-select all left columns

Search for columns...

Select types

6 of 6 columns selected

Select allDeselect all

units	Double	✓
transaction_date	Timestamp	✓
customer_id	String	✓
product_id	String	✓
transaction_id	String	✓
variation_id	String	✓

☐ Auto-select all right columns

Search for columns...

Select types

4 of 5 columns selected

Select allDeselect all

products_price	Double	✓
products_product_name	String	✓
products_category	String	✓
products_variation_id	String	✓
products_product_id	String	

Chaining Joins: Join Transactions x Products with Customers

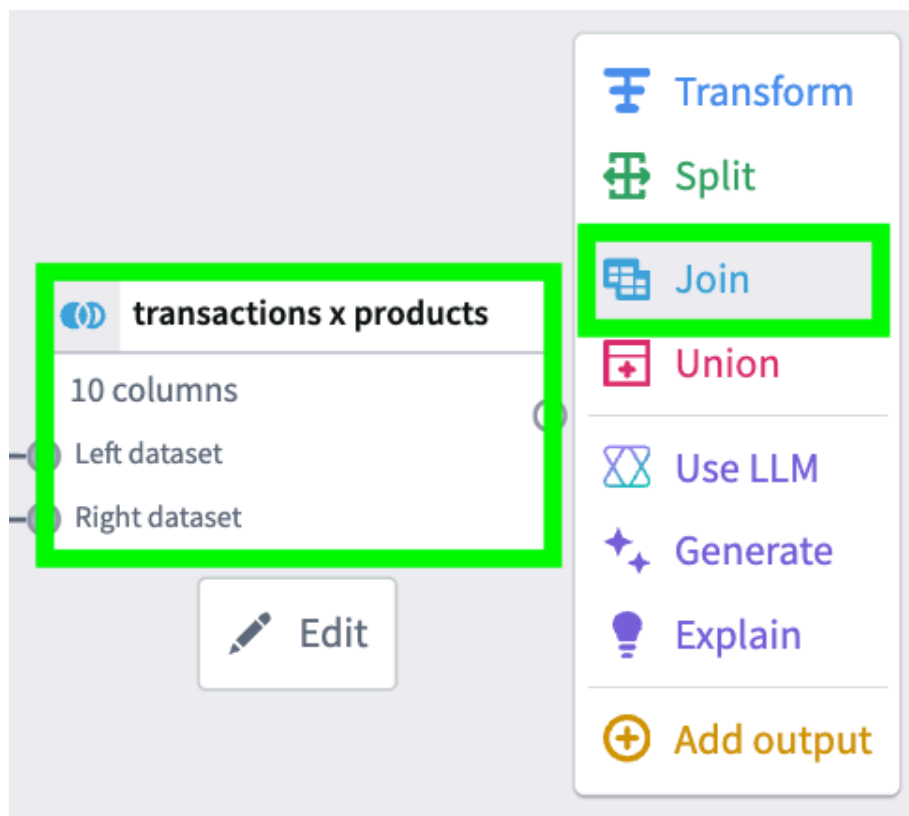
Lesson 15 / 25 · [原文链接](#)

Chaining joins is the process of combining multiple datasets sequentially, with each join operation using the output of the previous join as an input. This technique is commonly used when you need to integrate data from several sources to create a comprehensive and operationally valuable dataset.

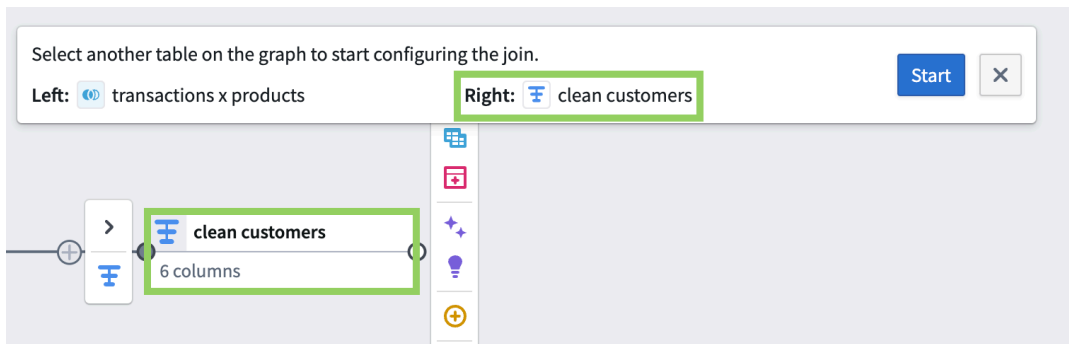
In your current example, you have created the **transactions x products** dataset by joining the two, and you now need to integrate the **customers** data to create a complete view of the total revenue for each customer.

Step 1: Join transactions x products with clean customers

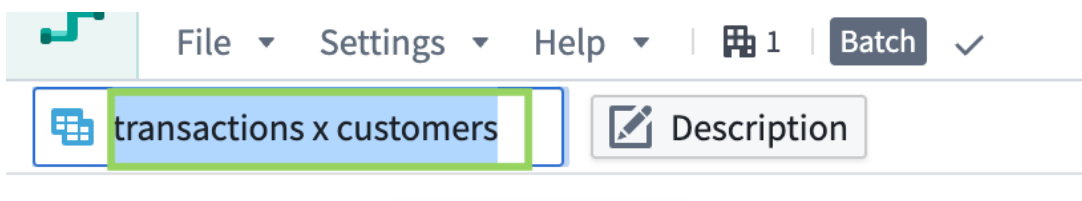
1. Select the **transactions x products** node and choose the **Join** option on the right hand side.



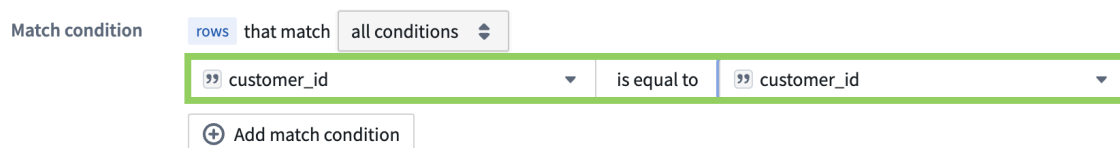
2. Choose the **clean customers** node to join with. Click **Start**.



3. Give the join a relevant name, for example **transactions x products x customers**, or just **transactions x customers**.



4. Select **customer_id is equal to customer_id** as the match condition.



Step 2: Verify updates and apply

1. Ensure your updates match the screenshot below.
2. Click **Apply** and **Close**.

transactions x customer

Description

AppliedClose

Join type

Left join

Joins two datasets together, keeping all rows from the left table and only rows which satisfy the provided condition from the right table.

Input tables

transactions x products

Swap

clean customers

Match condition

rows that match

all conditions

BasicAdvanced

customer_id

is equal to

customer_id

Add match condition

Select columns

Left: transactions x products

Right: clean customers

Auto-select all left columns

Search for columns...

Select types

10 of 10 columns selected

Selected

Not selected

Select all

Deselect all

transaction_date

Timestamp

✓

transaction_id

String

✓

customer_id

String

✓

variation_id

String

✓

product_id

String

✓

products_price

Double

✓

products_product_name

String

✓

products_category

String

✓

Prefix right columns

Auto-select all right columns

Search for columns...

Select types

5 of 6 columns selected

Selected

Not selected

Select all

Deselect all

address

String

✓

registration_date

Timestamp

✓

registration_date_tz

String

✓

email

String

✓

customer_id

String

name

String

✓

Add a Dataset Output

Lesson 16 / 25 · [原文链接](#)

The nodes in your pipeline so far are each intermediate transformations. Materializing outputs refers to the process of storing the results of a data transformation or join operation as a new dataset, which can be reused for other use cases and analyses within Foundry.

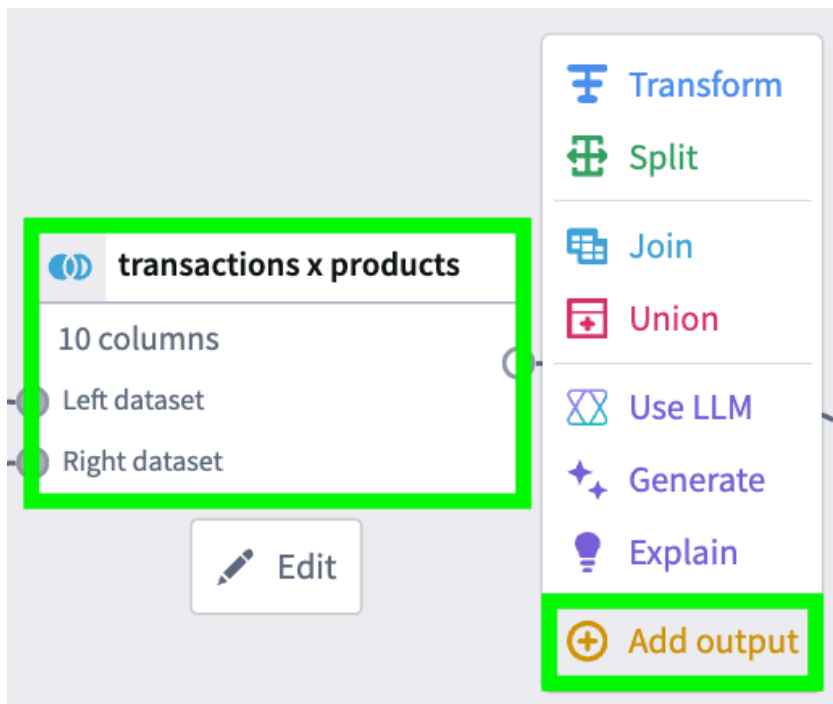
Materializing outputs is particularly beneficial when working with large or complex data transformations, as it allows you to break down your pipelines to encapsulate the results. Materializing outputs can improve the maintainability and traceability of your data pipeline, so you can track the lineage of each dataset and understand how it was derived.

Materialized outputs can not be further used as inputs to other transformations, and they represent the outcome of your pipeline. For more information on Pipeline Builder outputs, please reference the [Foundry Documentation](#).

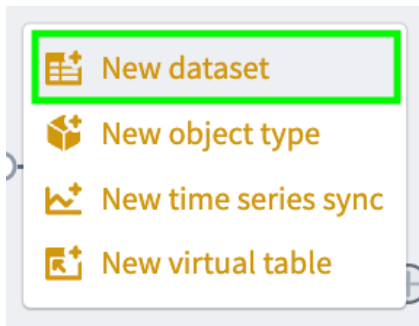
In your current example, you will materialize the outputs of both the **transactions x products** join and the **transactions x products x customers** join, since they will be operationally valuable for various use cases further down the line.

Step 1: Add a new dataset output to transactions x products

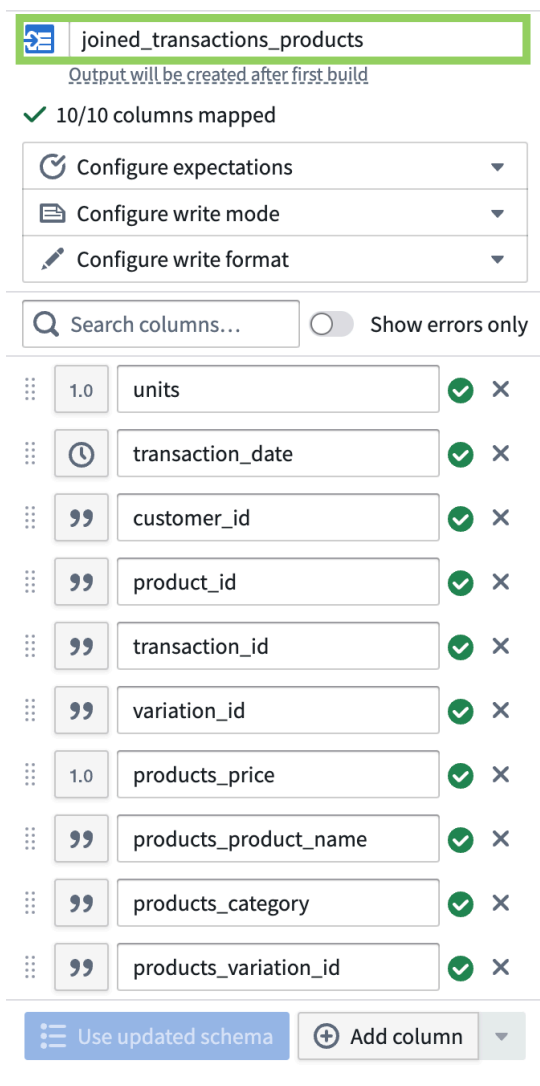
1. Navigate to the **transactions x products** node and select the **Add output** option in the list.



2. Select **New dataset**.

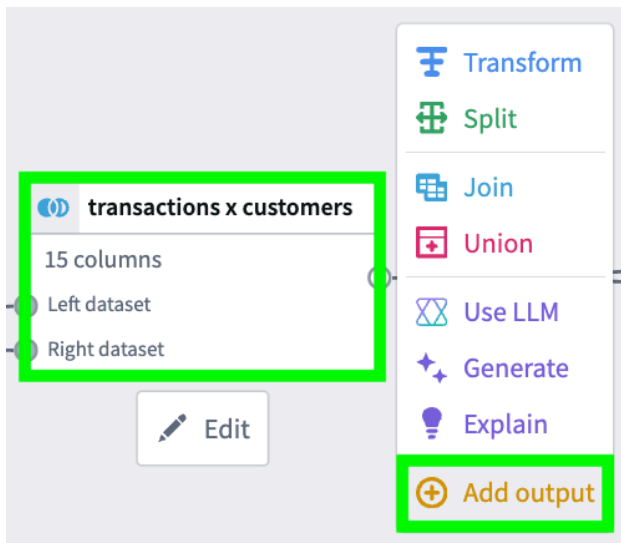


3. Give the generated dataset a relevant name on the right hand side.

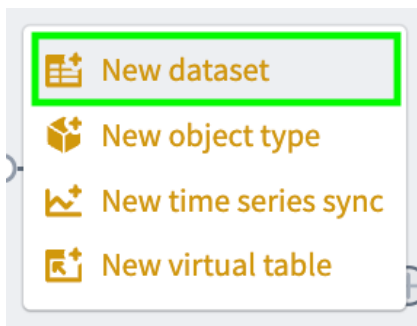


Step 2: Add a new dataset output to transactions x customers

1. Navigate to the **transactions x customers** node and select the **Add output** option in the list.



2. Choose **New dataset**.



3. Give the dataset a relevant name.

← Back to outputs

joined_transactions_products_customers

Output will be created after first build

✓ 15/15 columns mapped

Configure expectations

Configure write mode

Configure write format

Search columns...

Show errors only

1.0	units	✓	×
🕒	transaction_date	✓	×
”	customer_id	✓	×
”	product_id	✓	×
”	transaction_id	✓	×
”	variation_id	✓	×
1.0	products_price	✓	×
”	products_product_name	✓	×
”	products_category	✓	×
”	products_variation_id	✓	×
”	address	✓	×
🕒	registration_date	✓	×
”	registration_date_tz	✓	×
”	email	✓	×
”	name	✓	×

Use updated schema

Add column

Step 3: Save all changes and deploy the pipeline

1. Click **Save**
2. Click **Deploy** and note the pop-up stating that a build is starting.

Once the pipeline is deployed, your outputs will be materialized into Foundry resources and will be created in the folder of the pipeline.

Main

Save

Propose

Deploy

Deploy this pipeline

Update pipeline logic and build target outputs.

Last deployment

Succeeded

Feb 14, 2024, 4:20 PM

Deploy settings

Errors

Select outputs to build

Deploying the pipeline will update logic for all outputs, but only the selected outputs will be built.

>

☒

All

View changes

Deploy pipeline

Identify a Bad Join: Drill into Transactions x Products

Lesson 17 / 25 · [原文链接](#)

In some cases, a bad join occurs when two datasets are combined incorrectly, leading to inaccurate, duplicate, or missing data in the resulting dataset. This can significantly impact the quality of the insights derived from the data and, ultimately, the decisions made based on those insights.

When working with data, it's crucial to understand the relationships between different datasets and the keys used to join them. In many cases, a bad join can be the result of using the wrong key or not considering all relevant keys in the join condition. This can lead to a dataset that is either too large, due to duplicate entries, or too small, due to missing data.

It is a recommended practice to analyze the outputs of your joins and the techniques introduced in the sections before, such as the Preview functionality, are a great starting point for small datasets. In this section, however, you will leverage a different Foundry tool, Contour, to help analyze the joins that you created in the previous steps. In most cases, you'd be able to do your basic analyses in Preview, however Contour allows you to look through the entire data and discover bugs that would otherwise be impossible to find in Preview.

For more information about Contour as a Foundry tool, please refer to the [Foundry Documentation](#).

Step 1: Analyze the transactions x products result using Preview

1. Click on the **transactions x products** node and open the Preview tab at the bottom of the page. Notice that there are 172 rows in the output, whereas **cleaned transactions** only has 50 rows. The expectation would be that each transaction is enriched after the join, but not multiplied. This means that the join likely introduces multiplications.

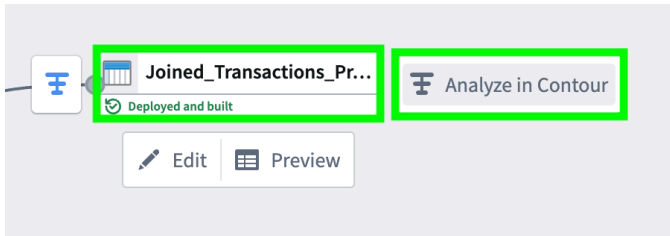
The screenshot shows a data pipeline in Foundry. The pipeline consists of three nodes: 'clean transactions' (6 columns), 'clean products' (10 columns), and 'transactions x products' (10 columns). The 'transactions x products' node is highlighted with a green box. Below the pipeline, the 'Preview' tab is open, showing a table with 172 rows. The table has columns: 'units' (Double), 'transaction_date' (Timestamp), 'customer_id' (String), 'product_id' (String), 'transaction_id' (String), and 'variation_id' (String). The table shows a list of transactions, with some rows having multiple entries for the same transaction date and product ID, indicating a bad join.

	units	transaction_date	customer_id	product_id	transaction_id	variation_id
1	610	2024-01-25T07:21:01.000Z	f9f95fe6-517a-4964-9	77d64cec-d43b-4115-a	02a10def-53bc-4eb8-a	53a47d0e-67a4-4241-8
2	386	2023-03-22T07:12:09.000Z	ac6ec763-0204-4634-9	f25805ba-083f-440a-a	04e2c878-0f65-4e6a-8	9e970f56-1dae-4d7a-9
3	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
4	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
5	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
6	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
7	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
8	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
9	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
10	12	2023-07-29T19:26:02.000Z	62470fcb-4b42-45d7-8	640a12fe-b227-4bfd-b	08a0d04c-e725-4f65-8	1405fac0-c74d-4690-9
11	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
12	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
13	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
14	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
15	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
16	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
17	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9
18	41	2023-02-22T08:37:31.000Z	1475707f-c65e-4d07-a	330db213-a83e-4a7c-b	11b8f83a-31ce-4e8d-9	65d3023f-a581-4699-9

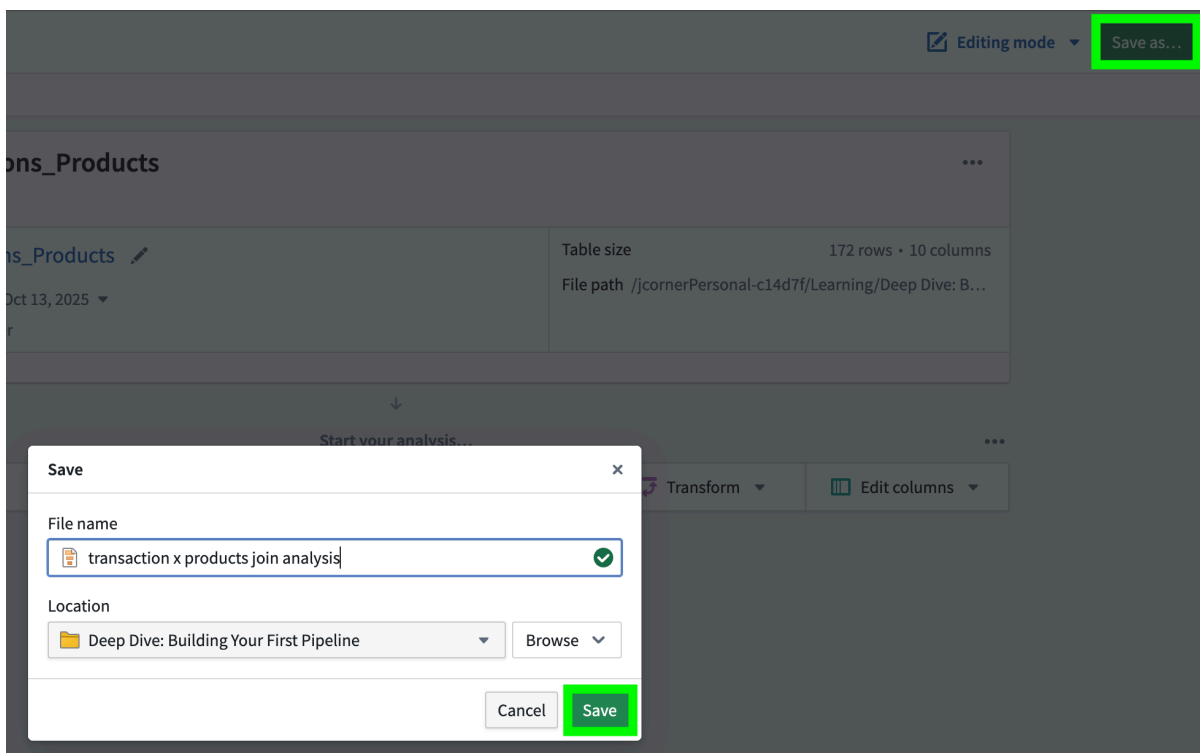
2. You can double check the number of rows in `cleaned_transactions` using Preview as well.
-

Step 2: Open the materialized dataset using Contour

1. From the transactions x products dataset, click **Analyze in Contour**.



2. A new tab will open. Navigate to it and click **Save as** to save your Contour analysis in your working folder.



Step 3: Check for `transaction_id` primary key uniqueness

1. Hover the cursor over the downward pointing arrow just above *Start your analysis* to reveal three options: **Edit path**, **Insert board** and **Paste board**
2. Click **Insert board**.

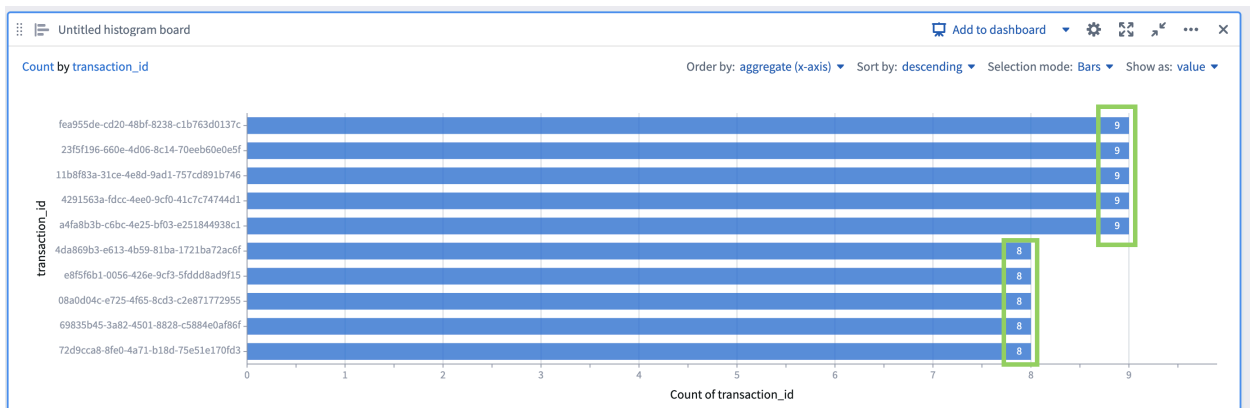
3. Search and select **Histogram**.

The screenshot shows the Palantir Foundry interface for the dataset **joined_transactions_products**. The dataset details include a table size of 172 rows and 10 columns, and a file path. Below the dataset information, there are buttons for 'Show data', 'Edit path', 'Insert board', and 'Paste board'. A search bar is active with the text 'his', and a dropdown menu shows the 'Histogram' option selected. To the right, a preview of a histogram titled 'Count by fruit' is visible, showing horizontal bars of varying lengths.

3. Plot the count of the **transaction_id**. This board will then display how many times each primary key occurs in the dataset. Any count more than 1 denotes duplication.

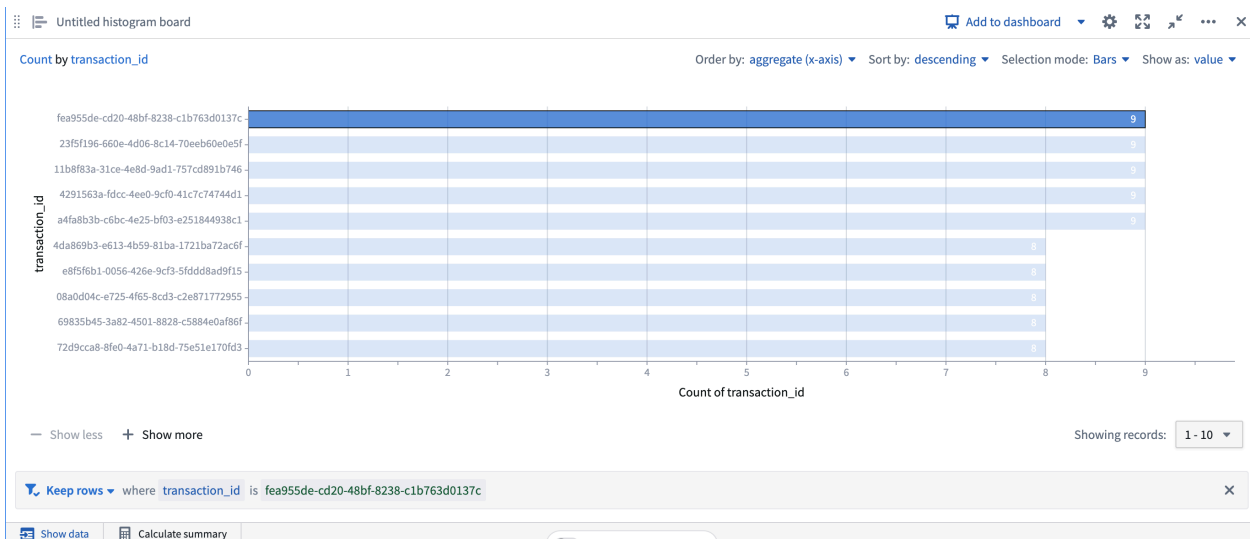
The screenshot shows the 'Configure Board' dialog box. The 'Y-Axis' section is set to 'Column'. The 'X-Axis' section is set to 'Count' of 'transaction_id'. A search bar is active with the text 'transaction_id', and a dropdown menu shows the 'transaction_id' option selected. The 'Close' and 'Compute' buttons are visible at the bottom right.

4. Notice that some **transaction_ids** appear up to 9 times, meaning that there was duplication introduced by the join.



Step 4: Drill into one of the duplicated ids.

1. Click on the top result. This will automatically filter to the selected values.



Step 5: Open the data and notice the duplication on product_id but the uniqueness of products_variation_id

1. Open the filtered data, by clicking the **Show data** button on the bottom left.



2. Notice that all columns have the same value, except for the ones prefixed with **products_**.

Data for
Untitled histogram board
Calculate summary
Export
products_variation_id
Filter

65d3023f-a581-4699-98bb-6547608af263, ab48f562-bb03-4fd7-af81-4c84e97e0f76, or 7 others in products_variation_id (Clear)

	transaction_date	customer_id	product_id	transaction_id	variation_id	products_price	products_prod...	products_categ...	products_variation_id
	Timestamp	String	String	String	String	Double	String	String	String
1	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	247	Oriental Fresh Sau...	Games	65d3023f-a581-4699-98bb...
2	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	839	Oriental Fresh Sau...	Outdoors	ab48f562-bb03-4fd7-af81...
3	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	809	Oriental Fresh Sau...	Clothing	0d3484b3-2464-4ee5-a769...
4	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	390	Oriental Fresh Sau...	Kids	d5d1ae64-9a99-4a89-b498...
5	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	878	Oriental Fresh Sau...	Movies	37130fa3-97d3-48df-aa03...
6	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	853	Oriental Fresh Sau...	Sports	a4612d11-eb16-4cc8-b492...
7	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	349	Oriental Fresh Sau...	Toys	61f23e84-8738-4386-81e7...
8	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	351	Oriental Fresh Sau...	Sports	4d28ff4c-4818-4d89-b92d...
9	Apr 03, 2023 9:30:26 PM +0...	1475707f-c65e-4d...	330db213-a83e-4a...	fea955de-cd20-48...	65d3023f-a5...	383	Oriental Fresh Sau...	Shoes	f8061c63-3d89-4c44-bf5f...

3. This means that the `product_ids` are not unique, and instead each product has a unique `variation_id` which is what introduced the duplication in the join.

Step 6: Return to the pipeline and fix the join

1. Return to your Pipeline Builder resource.
2. Change the match condition in the `transactions x products` join to be based off `variation_id`.
3. Apply the changes and return to the main screen with dataset nodes.

transactions x products

Description

Apply

Join type

Left join

Left joins left and right dataset inputs together.

Input tables

clean transactions

Swap

clean products

Match condition

rows that match

all conditions

variation_id

is equal to

variation_id

Add match condition

Create an Operationally Valuable Dataset

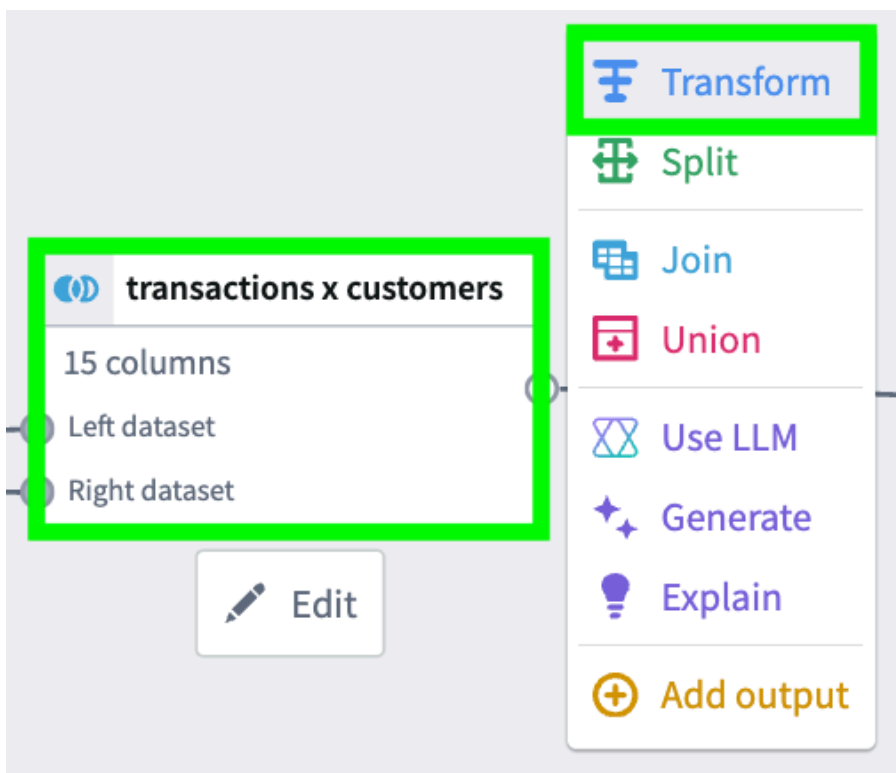
Lesson 18 / 25 · [原文链接](#)

In this section, you will finally use the data to create an operationally valuable dataset, representing the customer lifetime value. Customer lifetime value is a metric that represents the total net profit a company can expect to earn from a customer over the entire duration of their relationship.

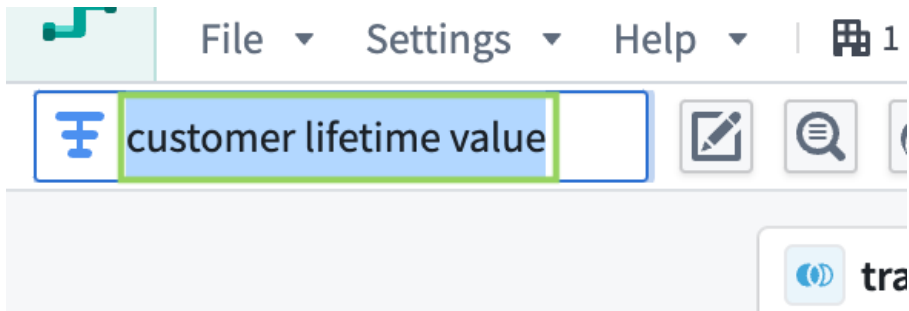
Using the combined dataset from the **transactions (x products) x customers** join, you will leverage Pipeline Builder transforms to aggregate the total revenue generated by each customer and derive their lifetime value. This is, of course, just one example of how you can transform the joined data into valuable insights that can drive business decisions.

Step 1: Transform the transactions x customers dataset

1. Select the **transactions x customers** node and choose the **Transform** option on the right hand side.

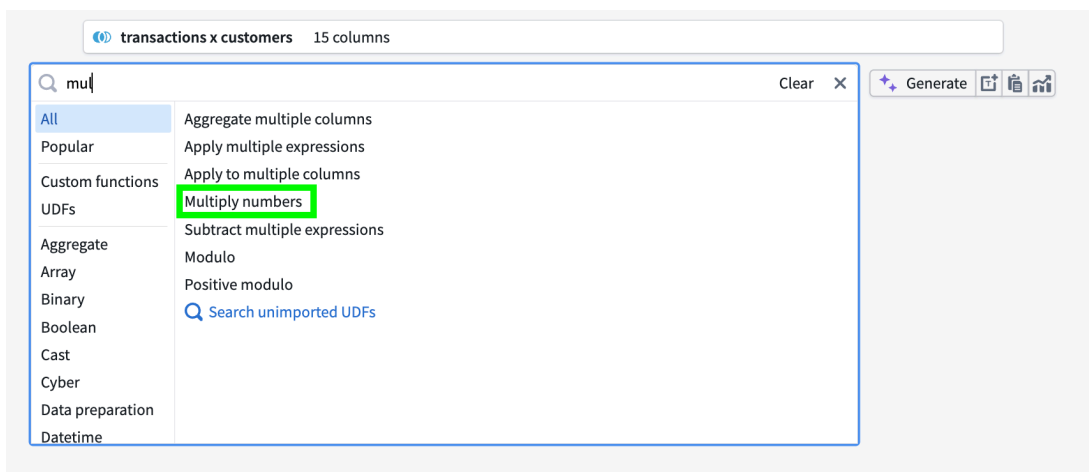


2. Give the transform a relevant name like `customer lifetime value`.

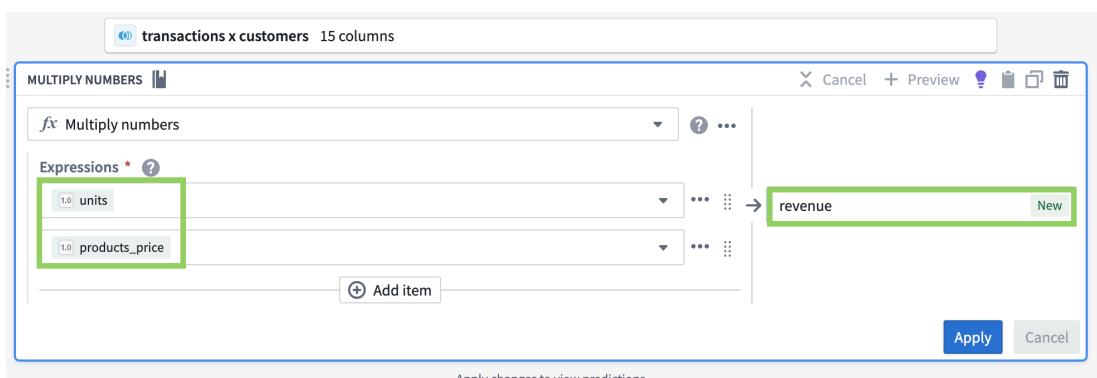


Step 2: Compute the revenue per transaction

1. Select the **Multiply numbers** transform. The revenue generated by each transaction is the number of units sold multiplied by the price of a unit.



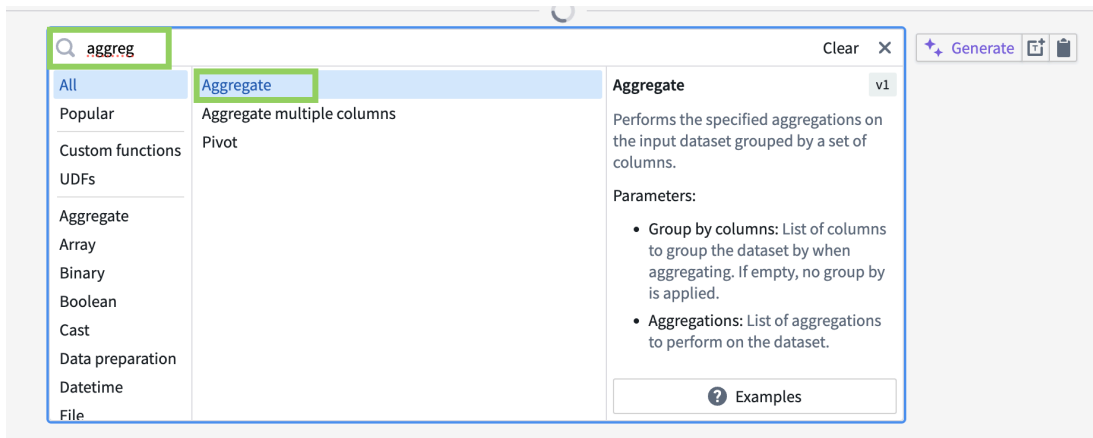
2. Select the **units** and **products_price** columns, and name the output column **revenue**.



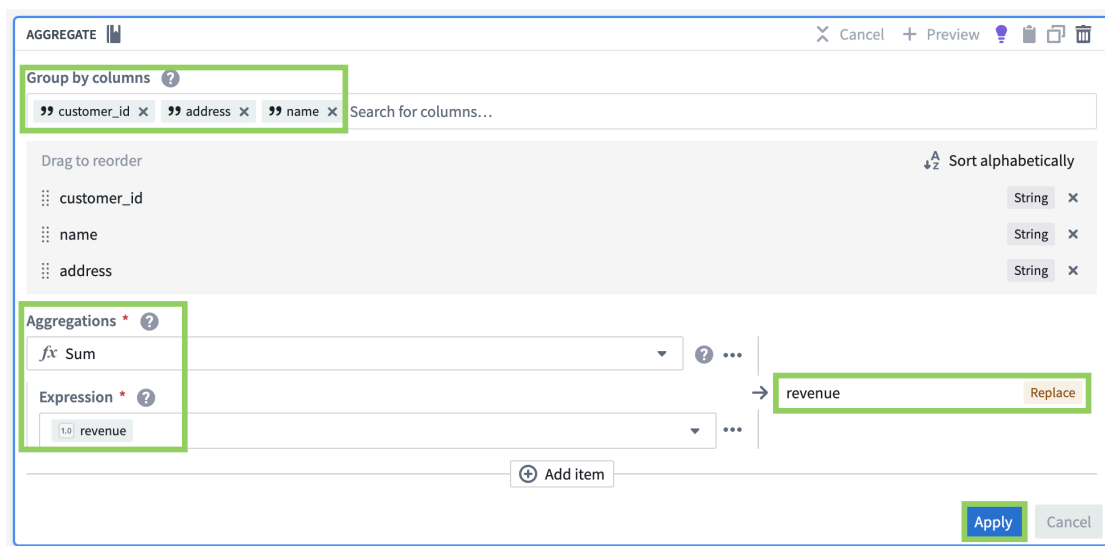
3. Click **Apply**.

Step 3: Aggregate per customer

1. Search for the **Aggregate** transform.



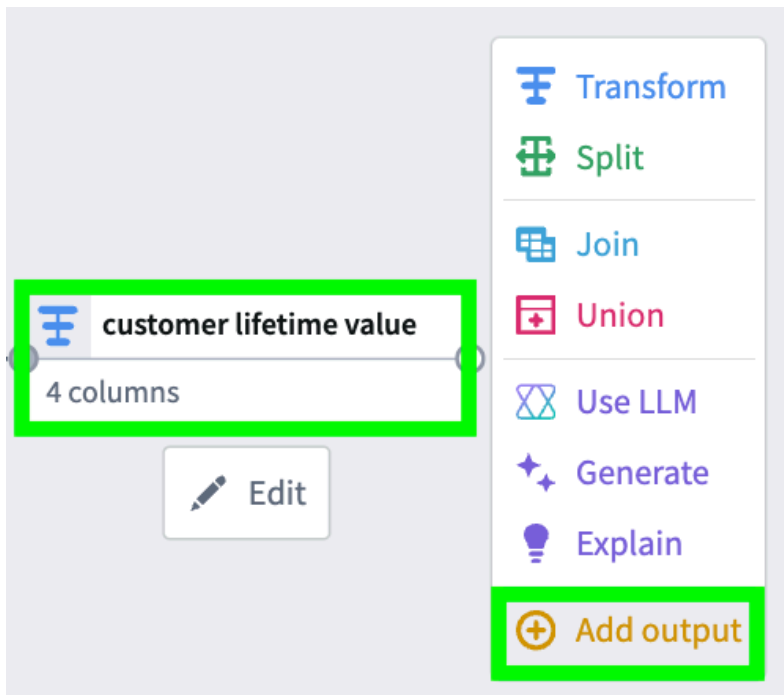
2. Sum the **revenue** column per **customer**, and keep some important columns containing customer details.



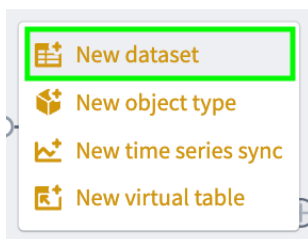
3. Apply all changes and return to the graph.

Step 4: Add a new dataset output to customer lifetime value

1. Navigate to the **customer lifetime value** node and select the **Add output** option from the list.



2. Choose **New dataset**.



3. Give the output dataset a relevant name.

[← Back to outputs](#)  


Output will be created after first build

 4/4 columns mapped

 Configure expectations

 Configure write mode

 Configure write format

☐ Show errors only

		customer_id		
		name		
		address		
	1.0	revenue		

 Use updated schema

 Add column 

4. Save and deploy the pipeline.

Introduction

Lesson 19 / 25 · [原文链接](#)

The previous sections have laid the groundwork with the fundamental techniques on how to construct a data pipeline in Foundry. However, it is important to recognize that the reality of data processing and engineering often presents scenarios more intricate than the example in this training path. In this section, you will explore some advanced considerations and best practices that will help you handle real-life data complexities.

This section will highlight additional Pipeline Builder capabilities, such as streaming pipelines for real-time data processing and directly outputting to Foundry's ontology. Additionally, this section will cover key areas to consider as you advance your pipeline development, namely the use of custom User-Defined Functions (UDFs) for those instances when pre-built transforms don't fit your requirements, strategies for segmenting your pipeline and determining when to materialize outputs, and ways of maintaining your pipelines long-term.

This course is being actively improved - we would love to hear your feedback at training-feedback@palantir.com

Other Pipeline Builder Capabilities

Lesson 20 / 25 · [原文链接](#)

Pipeline Builder stands out for its versatility, allowing you to create sophisticated data flows beyond the basic tasks of data cleaning and preprocessing.

Streaming

One of the notable features of Pipeline Builder is its support for **streaming pipelines**, suitable for time-sensitive scenarios, where data needs to be processed with minimal delay. Streaming pipelines process data in (almost) real-time as it is ingested, being processed on average in under 15 seconds.

For more information about how to create a streaming pipeline, please refer to the [Foundry Streaming Documentation](#).

The screenshot shows the Foundry Pipeline Builder interface for a 'Temperature Stream'. The interface is divided into several sections:

- Throughput:** Two options are shown: 'Normal' (Recommended) and 'Very high'. 'Normal' is suitable for up to 5MB/s of throughput with row order guaranteed, lowest compute cost, and suitable for up to 5MB/s of throughput. 'Very high' is best for data over 5MB/s of throughput but has order not guaranteed, higher compute cost, and is best for data over 5MB/s of throughput. A slider below shows a range from 10MB/s to 50.
- Schema:** A table lists columns: 'sensor_id' (String, 99, nullable), 'temperature' (Double, 123, nullable), and 'Add a column...'. To the right, the 'SCHEMA JSON' is displayed:

```
{  "type": "STRING",  "nullable": false,  "name": "sensor_id",  "customMetadata": {}}, {  "type": "DOUBLE",  "nullable": false,  "name": "temperature",  "customMetadata": {}}
```
- Keys:** A section titled 'Keys' with an 'Optional' tag. It explains that optionally selecting a set of key columns guarantees ordering between unique IDs. Rows will not be guaranteed to be stored in order unless the value of their keys are the same or the stream is configured with a single partition only. A search bar is provided.

At the bottom, there are status indicators: 'Partitions: 1', '2 columns', and '0 keys'. A 'Create stream' button is located in the bottom right corner.

Outputting to ontology

In addition to its data processing capabilities, Pipeline Builder also provides options for the destination of the output. While the creation of datasets is a common outcome, **Pipeline**

Builder can also direct the output straight into an ontology. In Foundry, ontologies represent a comprehensive knowledge graph that encapsulates the relationships and connections between various data entities. Directing output to an ontology facilitates the immediate use of structured and relationally rich data for complex analyses and applications. For more information on how to output to the ontology, please refer to the [Foundry Ontology Outputs Documentation](#).

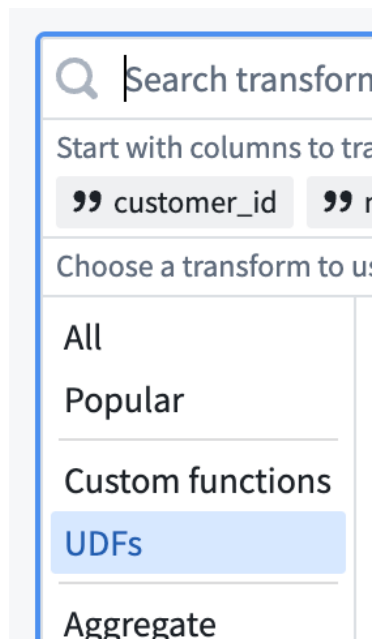
While this course primarily introduces the foundational elements of data pipeline creation, it's important to be aware that Pipeline Builder's functionality extends to advanced features such as streaming and ontology outputs. These features offer pathways to integrate your data pipeline more deeply with Foundry's extensive data management ecosystem.

User-Defined Functions (UDFs)

Lesson 21 / 25 · [原文链接](#)

The pre-built data transformations in Pipeline Builder are powerful, but they may not meet all the specific requirements of every data engineering task. As an advanced capability of Pipeline Builder, User Defined Functions (UDFs) can be leveraged in your pipeline to define custom functionality written in a programming language supported by Foundry.

For more information on how to create a UDF, please refer to the [Foundry Documentation](#). After [publishing a UDF to Pipeline Builder](#), you can leverage it in your transforms through the Pipeline Builder boards. Once published, a UDF cannot be unpublished.



Best Practices for UDFs

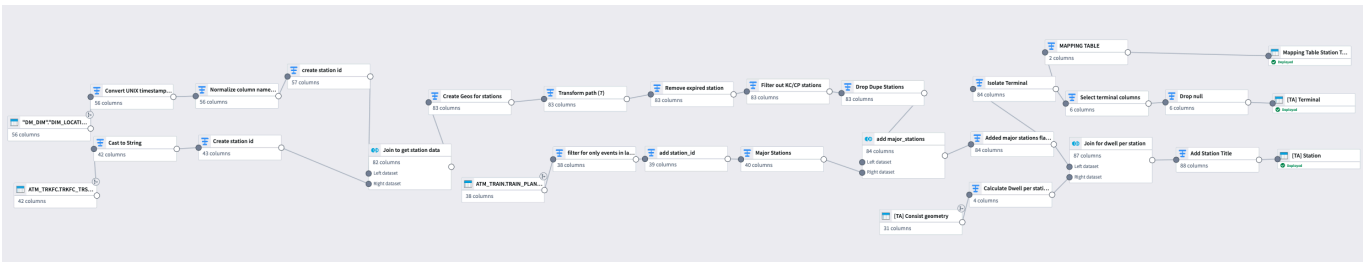
- **Identify the need for UDFs.** Before creating a UDF, ensure that the operation you need cannot be accomplished with the existing built-in transformations. UDFs should be a last resort due to their potential impact on performance and maintainability.
- **Minimize usage.** Only use UDFs when necessary. Leverage built-in functions whenever possible as they are typically optimized for performance.
- **Keep it simple.** Write UDFs that are simple, perform a single task, and minimize overlap with other built-in transformations. Complex UDFs are harder to maintain and can be less efficient.

Best Practices: Segmenting your Pipeline and Materializing Outputs

Lesson 22 / 25 · [原文链接](#)

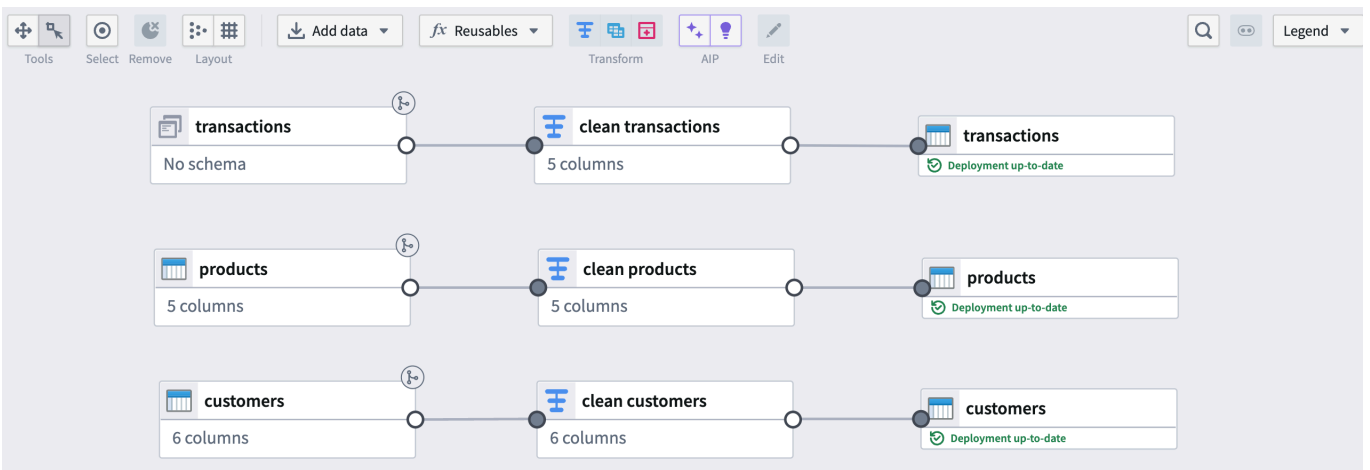
In practice, your data pipeline may require a vast number of inputs, multiple intermediate steps, or tight connectedness between siloed data points. As the number of nodes grows in your pipeline, the graphical representation in Pipeline Builder can swiftly become overwhelming. In scenarios where your pipeline exceeds 100 nodes, you may struggle with navigation and even encounter difficulties in loading the graph. The key to maintaining manageability lies in strategic segmentation of your pipeline and which outputs you decide to materialize.

A pipeline that defers the materialization of outputs until the final stages is optimized for latency. Each output is effectively deployed and built into one single step from the pipeline inputs, so if you have a pipeline that goes through the typical stages of “raw” to “clean” to “joined” to “ontology”, the ontology outputs require only one step of chained intermediary transforms to be deployed and materialized into a Foundry resource. A common approach for organizing your pipeline when optimizing for latency is to split it by use case, where each pipeline generates end to end transformations powering each independent workflows in Foundry.

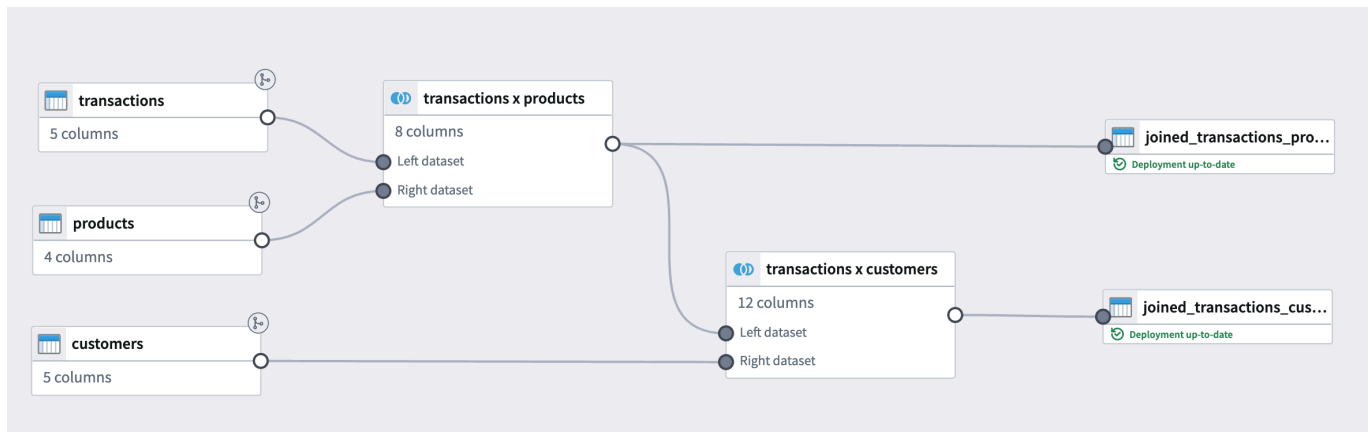


end-to-end use case pipeline

Alternatively, in the context of a large-scale use case with intertwined data points, you may prefer to optimize for extensibility and readability, as opposed to latency. In this case, consider dividing your project into distinct segments, each with its own Pipeline Builder instance, such as one for "cleaning", one for "joining", one for "ontology", and so forth. By materializing the outputs at the conclusion of each segment and then using those outputs as inputs for subsequent stages, you maximize reusability and clarity, at the cost of build latency.



clean pipeline



joined pipeline

Regardless of your approach, you can think of segmenting your pipeline similarly to software development, where code is modularized rather than amalgamated into a single file. By applying similar principles to your data pipeline, you can ensure that each segment is focused, manageable, and tailored to a particular aspect of your data processing.

As a rule of thumb, you can refer to the below best practices:

- **Modularize your pipeline** just as you would with code, dividing your pipelines into logical sections that can be developed, tested, and debugged independently.
- **Materialize judiciously** at key points to create natural breakpoints and to capture intermediary states that might be reusable in other contexts.
- **Test segments** at key points to identify bad joins, wrong aggregations, or incomplete assumptions about the data, to catch errors quickly before they trickle through your pipeline.

By implementing these best practices, you can create a pipeline that is not only optimized for performance but also for development and maintenance, ensuring that your data processing workflows remain robust, scalable, and understandable.

Pipeline Maintainability

Lesson 23 / 25 · [原文链接](#)

As you grow your data infrastructure in Foundry, maintaining the performance and accuracy of your pipelines is essential. Proper maintenance ensures that the pipeline remains reliable, efficient, and up-to-date with evolving data structures and business requirements. Future trainings will focus more broadly on comprehensive maintenance strategies, so here is a highlight of the key areas you might want to dive deeper on.

Each of these areas plays a vital role in maintaining a robust and effective data pipeline. By leveraging Pipeline Builder out-of-the-box functionality and adhering to best practices, you can ensure that your pipeline remains a reliable asset for data-driven decision-making. For a more comprehensive understanding of pipeline maintenance, consider exploring the dedicated training modules available on these topics.

Scheduling

A key next step after implementing the logic of your data processing is implementing a scheduling strategy, to ensure data is refreshed at regular intervals, keeping it current with the source and relevant for the workflows. Pipeline Builder offers out-of-the-box scheduling capabilities that can be configured directly in your pipeline to meet your specific needs. For a detailed guide on setting up and managing pipeline schedules, refer to the [Foundry Scheduling Documentation](#). In the documentation, you will also find a variety of [Scheduling Best Practices](#) to consider.

← Back to schedules list

Schedule name...

Schedule description...

Select scheduled outputs

customer_lifetime_value

×

None selected

+ Add

When to build

When specific datasets update

At a specific time

Select dataset nodes on graph and click "+Add" button to add to this list

None selected

+ Add

← Back to schedules list

Schedule name...

Schedule description...

Select scheduled outputs

customer_lifetime_value

×

None selected

+ Add

When to build

When specific datasets update

At a specific time

At 12:00pm (Europe/London)

+ Suggest

Run every

1

Day

▼

At

12 : 00

PM

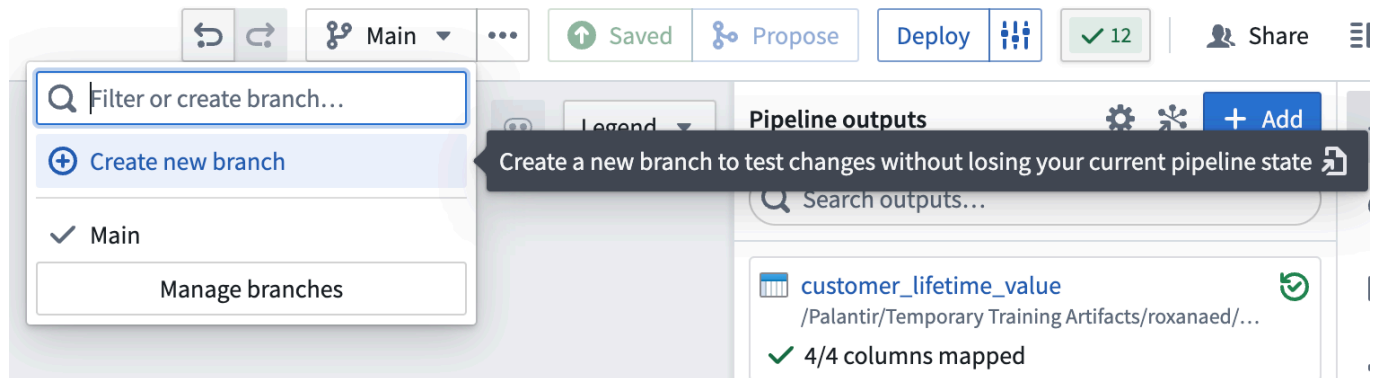
⬆️⬆️

London (GMT) +00:00

▼

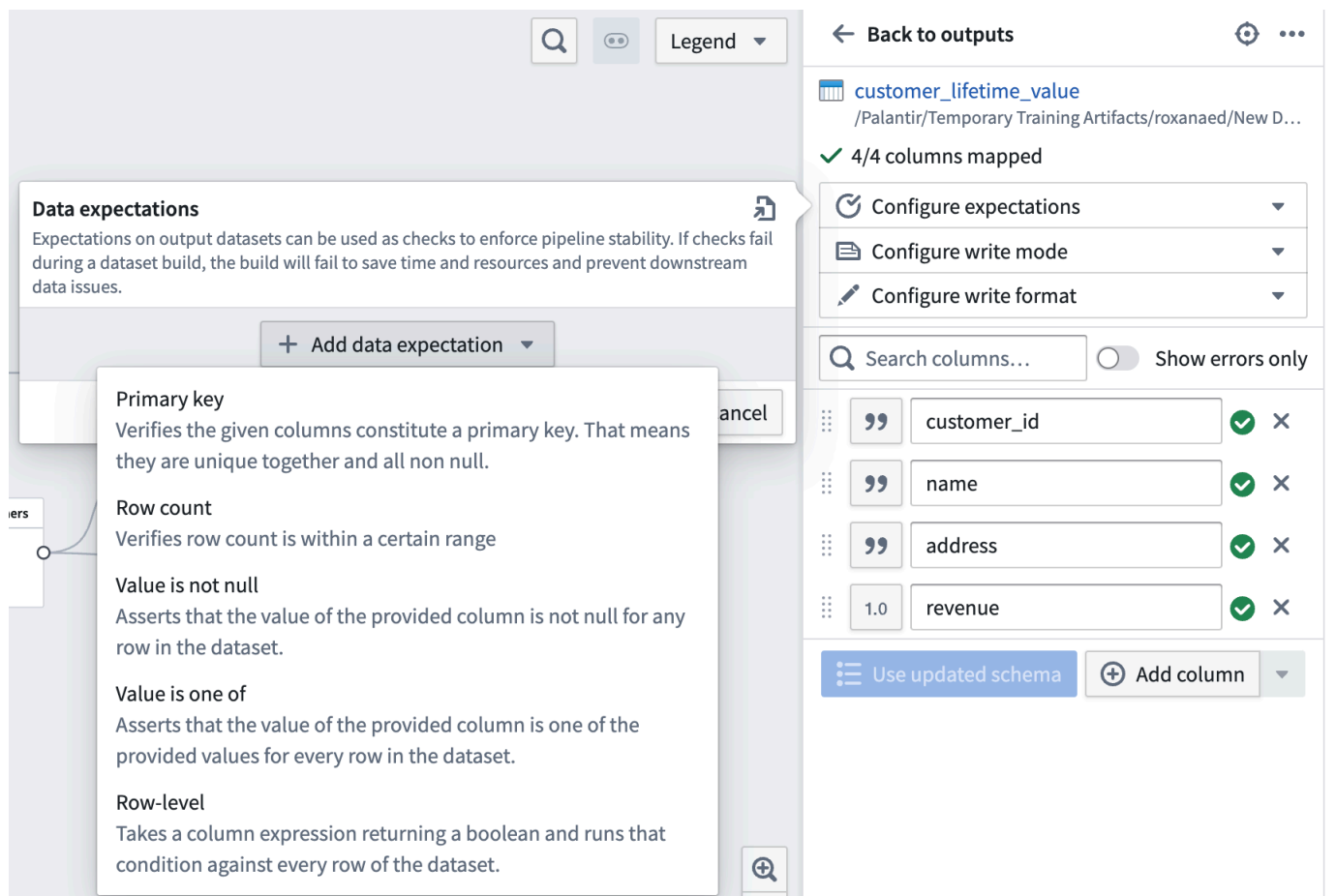
Branching

Once your pipeline is used across multiple workflows, enforcing branching is an important practice for maintaining a stable data pipeline while iterating on new features or updates. Learn more about branching strategies and how to implement branching in Pipeline Builder by visiting the [Foundry Branching Documentation](#).



Data Expectations

Setting data expectations, or "data quality checks," helps in proactively monitoring the health of your data. These checks can validate data against predefined criteria, such as data type constraints, value ranges, or uniqueness, to catch anomalies early. Foundry's data expectations tools are discussed in more detail in the [Foundry Data Expectations Documentation](#).



Incremental Transforms

For efficiency, especially with large volumes of data, incremental transforms process only the data that has changed since the last pipeline run, rather than reprocessing the entire dataset. This can significantly reduce the computational load and speed up the time-to-insight. Foundry's capabilities for incremental processing are detailed in the [Foundry Incremental Transforms Documentation](#).

Summary

Lesson 24 / 25 · [原文链接](#)

Congratulations, you've completed the Deep Dive: Building Your First Pipeline course! You've now learned the basics of Pipeline Builder, Foundry's no-code pipelining tool.

Looking to continue learning?

Explore the [Data Engineer Training Track](#) for more courses and resources related to this course.

Loading...

Lesson 25 / 25 · [原文链接](#)

Loading...